Programación multiproceso.

La programación multiproceso es una técnica utilizada para ejecutar múltiples procesos de manera simultánea en un sistema informático. A diferencia de la programación secuencial, donde las tareas se ejecutan una tras otra, en el multiproceso, varias tareas pueden ejecutarse en paralelo, lo que permite un uso más eficiente de los recursos del sistema, como el procesador.

En el ámbito del desarrollo de aplicaciones, especialmente en la asignatura de Programación de Servicios y Procesos del ciclo formativo de Desarrollo de Aplicaciones Multiplataforma (DAM), esta técnica es esencial para aprender cómo mejorar el rendimiento de programas que necesitan manejar múltiples tareas simultáneamente, como servidores web, aplicaciones de bases de datos o videojuegos.

Conceptos Clave:

- Proceso: Es una instancia en ejecución de un programa. Cada proceso tiene su propio espacio de memoria y puede ejecutar tareas de forma independiente.
- Multiproceso: Se refiere a la capacidad de un sistema de ejecutar varios procesos al mismo tiempo, utilizando múltiples núcleos del procesador o incluso distribuyendo la carga en varios procesadores.
- Concurrencia: Se refiere a la capacidad de gestionar múltiples tareas en diferentes momentos de tiempo, lo que permite simular la ejecución paralela en sistemas de un solo procesador.
- Paralelismo: Se refiere a la ejecución simultánea de múltiples tareas en varios procesadores o núcleos de CPU, mejorando significativamente el rendimiento.

Ventajas del Multiproceso:

- Mejora del rendimiento: Al dividir el trabajo entre varios procesos, se puede reducir el tiempo total de ejecución de una tarea.
- Aprovechamiento de recursos: Los sistemas actuales cuentan con procesadores multinúcleo, y la programación multiproceso permite aprovecharlos al máximo.
- Resiliencia: Si un proceso falla, los demás pueden seguir ejecutándose sin afectar el sistema global.

Desafíos del Multiproceso:

- Sincronización: Controlar el acceso a los recursos compartidos entre varios procesos es crítico para evitar conflictos o resultados incorrectos.
- Consumo de recursos: Ejecutar múltiples procesos simultáneamente puede consumir más memoria y recursos del sistema, lo que requiere una buena planificación.

En resumen, la programación multiproceso es una habilidad clave para el desarrollo de aplicaciones eficientes y escalables, particularmente en el contexto de servicios y procesos, donde es necesario manejar múltiples conexiones o solicitudes de manera eficiente.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Introducción: Aplicaciones, Ejecutables y Procesos.

> **¿Aplicaciones, ejecutables y procesos? ¿Son lo mismo o cosas distintas? ¿Tres palabras distintas para lo mismo?**

A simple vista, parece que con los términos aplicación, ejecutable y proceso nos estamos refiriendo a lo mismo. Pero, no olvidemos que en los módulos de primero hemos aprendido a diferenciarlos.

Una aplicación es un tipo de programa informático, diseñado como herramienta para resolver de manera automática un problema específico del usuario.

Debemos darnos cuenta de que sobre el hardware del equipo, todo lo que se ejecuta son programas informáticos, que, ya sabemos, que se llama software. Con la definición de aplicación anterior, buscamos diferenciar las aplicaciones, de otro tipo de programas informáticos, como pueden ser: los sistemas operativos, las utilidades para el mantenimiento del sistema, o las herramientas para el desarrollo de software. Por lo tanto, son aplicaciones, aquellos programas que nos permiten editar una imagen, enviar un correo electrónico, navegar en Internet, editar un documento de texto, chatear, etc.

Recordemos, que un programa es el conjunto de instrucciones que ejecutadas en un ordenador realizarán una tarea o ayudarán al usuario a realizarla. Nosotros, como programadores y programadoras, creamos un programa, escribiendo su código fuente, con ayuda de un compilador, obtenemos su código binario o interpretado. Este código binario o interpretado, lo guardamos en un fichero. Este fichero, es un fichero ejecutable, llamado comúnmente: ejecutable o binario.

> **Un ejecutable es un fichero que contiene el código binario o interpretado que será ejecutado en un ordenador.**

Ya tenemos más clara la diferencia entre aplicación y ejecutable. Ahora, ¿qué es un proceso?

De forma sencilla, un proceso, es un programa en ejecución. Pero, es más que eso, un proceso en el sistema operativo (SO), es una unidad de trabajo completa; y, el SO gestiona los distintos procesos que se encuentren en ejecución en el equipo. En siguientes apartados de esta unidad trataremos más en profundidad todo lo relacionado con los procesos y el SO. Lo más importante, es que diferenciemos que un ejecutables es un fichero y un proceso es una entidad activa, el contenido del ejecutable, ejecutándose.

Un proceso es un programa en ejecución.

> **Un proceso es un programa en ejecución.**

Un proceso existe mientras que se esté ejecutando una aplicación. Es más, la ejecución de una aplicación, puede implicar que se arranquen varios procesos en nuestro equipo; y puede estar formada por varios ejecutables y librerías o bibliotecas.

> **Una aplicación es un tipo de programa informático, diseñado como herramienta para resolver de manera automática un problema específico del usuario. Al instalarla en el equipo, podremos ver que puede estar formada por varios ejecutables y librerías. Siempre que lancemos la ejecución de una aplicación, se creará, al menos, un proceso nuevo en nuestro sistema.**

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Ejecutables. Tipos.

> **¿Cómo reconocemos un ejecutable?**
> Antes hemos hecho referencia a ejecutables que contenían código binario o interpretado. ¿Sabes qué significa eso? ¿Son todos los ficheros ejecutables iguales?

En sistemas operativos Windows, podemos reconocer un fichero ejecutable, porque su extensión, suele ser .exe. En otros sistemas operativos, por ejemplo, los basados en GNU/Linux, los ficheros ejecutables se identifican como ficheros que tienen activado su permiso de ejecución (y no tienen que tener una extensión determinada).

Según el tipo de código que contenga un ejecutable, los podemos clasificar en:

- **Binarios**. Formados por un conjunto de instrucciones que directamente son ejecutadas por el procesador del ordenador. Este código se obtiene al compilar el código fuente de un programa y se guarda en un fichero ejecutable. Este código sólo se ejecutará correctamente en equipos cuya plataforma sea compatible con aquella para la que ha sido compilado (no es multiplataforma). Ejemplos son, ficheros que obtenemos al compilar un ejecutable de C o C++.
- **Interpretados**. Código que suele tratarse como un ejecutable, pero no es código binario, sino otro tipo de código, que en Java, por ejemplo se llama bytecode. Está formado por códigos de operación que tomará el intérprete (en el caso de Java, el intérprete es la máquina virtual Java, Java Runtime Environment o JRE). Ese intérprete será el encargado de traducirlos al lenguaje máquina que ejecutará el procesador. El código interpretado es más susceptible de ser multiplataforma o independiente de la máquina física en la que se haya compilado.

Un tipo especial de ejecutables interpretados, son los llamados scripts. Estos ficheros, contienen las instrucciones que serán ejecutadas una detrás de otra por el intérprete. Se diferencian de otros lenguajes interpretados porque no son compilados. Por lo que los podremos abrir y ver el código que contienen con un editor de texto plano (cosa que no pasa con los binarios e interpretados compilados). Los intérpretes de este tipo de lenguajes se suelen llamar motores. Ejemplos de lenguajes de script son: JavaScript, php, JSP, ASP, python, ficheros .BAT en MS-DOS, Powershell en Windows, bash scripts en GNU/Linux, …

- **Librerías**. Conjunto de funciones que permiten dar modularidad y reusabilidad a nuestros programas. Las hemos incluido en esta clasificación, porque su contenido es código ejecutable, aunque ese código sea ejecutado por todos los programas que invoquen las funciones que contienen. El conjunto de funciones que incorpora una librería suele ser altamente reutilizable y útil para los programadores; evitando que tengan que reescribir una y otra vez el código que realiza la misma tarea. Ejemplo de librerías son: las librerías estándar de C, los paquetes compilados DLL en Windows; las API (Interfaz de Programación de Aplicaciones), como la J2EE de Java (Plataforma Java Enterprise Edition versión 2); las librerías que incorpora el framework de .NET; etc.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Gestión de procesos.

Como sabemos, en nuestro equipo, se están ejecutando al mismo tiempo, muchos procesos. Por ejemplo, podemos estar escuchando música con nuestro reproductor multimedia favorito; al mismo tiempo, estamos programando con NetBeans; tenemos el navegador web abierto, para ver los contenidos de esta unidad; incluso, tenemos abierto el Messenger para chatear con nuestros amigos y amigas.

Independientemente de que el microprocesador de nuestro equipo sea más o menos moderno (con uno o varios núcleos de procesamiento), lo que nos interesa es que actualmente, nuestros SO son multitarea; como son, por ejemplo, Windows y GNU/Linux. Ser multitarea es, precisamente, permitir que varios procesos puedan ejecutarse al mismo tiempo, haciendo que todos ellos compartan el núcleo o núcleos del procesador. Pero, ¿cómo? Imaginemos que nuestro equipo, es como nosotros mismos cuando tenemos más de una tarea que realizar. Podemos, ir realizando cada tarea una detrás de otra, o, por el contrario, ir realizando un poco de cada tarea. Al final, tendremos realizadas todas las tareas, pero para otra persona que nos esté mirando desde fuera, le parecerá que, de la primera forma, vamos muy lentos (y más, si está esperando el resultado de una de las tareas que tenemos que realizar); sin embargo, de la segunda forma, le parecerá que estamos muy ocupados, pero que poco a poco estamos haciendo lo que nos ha pedido. Pues bien, el micro, es nuestro cuerpo, y el SO es el encargado de decidir, por medio de la gestión de procesos, si lo hacemos todo de golpe, o una tarea detrás de otra.

En este punto, es interesante que hagamos una pequeña clasificación de los tipos de procesos que se ejecutan en el sistema:

- **Por lotes**. Están formados por una serie de tareas, de las que el usuario sólo está interesado en el resultado final. El usuario, sólo introduce las tareas y los datos iniciales, deja que se realice todo el proceso y luego recoge los resultados. Por ejemplo: enviar a imprimir varios documentos, escanear nuestro equipo en busca de virus,...
- **Interactivos**. Aquellas tareas en las que el proceso interactúa continuamente con el usuario y actúa de acuerdo a las acciones que éste realiza, o a los datos que suministra. Por ejemplo: un procesador de textos; una aplicación formada por formularios que permiten introducir datos en una base de datos; ...
- **Tiempo real**. Tareas en las que es crítico el tiempo de respuesta del sistema. Por ejemplo: el ordenador de a borde de un automóvil, reaccionará ante los eventos del vehículo en un tiempo máximo que consideramos correcto y aceptable. Otro ejemplo, son los equipos que controlan los brazos mecánicos en los procesos industriales de fabricación.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Gestión de procesos. Introducción.

En nuestros equipos ejecutamos distintas aplicaciones interactivas y por lotes. Como sabemos, un microprocesador es capaz de ejecutar miles de millones de instrucciones básica s en un segundo (por ejemplo, un i7 puede llegar hasta los 3,4 GHz). Un micro, a esa velocidad, es capaz de realizar muchas tareas, y nosotros (muy lentos para él), apreciaremos que solo está ejecutando la aplicación que nosotros estamos utilizando. Al fin y al cabo, al micro, lo único que le importa es ejecutar instrucciones y dar sus resultados, no tiene conocimiento de si pertenecen a uno u otro proceso, para él son instrucciones. Es, el SO el encargado de decidir qué proceso puede entrar a ejecutarse o debe esperar. Lo veremos más adelante, pero se trata de una fila en la que cada proceso coge un número y va tomando su turno de servicio durante un periodo de tiempo en la CPU; pasado ese tiempo, vuelve a ponerse al final de la fila, esperando a que llegue de nuevo su turno.

Vamos a ver cómo el SO es el encargado de gestionar los procesos, qué es realmente un programa en ejecución, qué información asocia el SO a cada proceso. También veremos qué herramientas tenemos a nuestra disposición para poder obtener información sobre los procesos que hay en ejecución en el sistema y qué uso están haciendo de los recursos del equipo.

Los nuevos micros, con varios núcleos, pueden, casi totalmente, dedicar una CPU a la ejecución de uno de los procesos activos en el sistema. Pero no nos olvidemos de que además de estar activos los procesos de usuario, también se estará ejecutando el SO, por lo que seguirá siendo necesario repartir los distintos núcleos entre los procesos que estén en ejecución.

¿No te gustaría ver cómo se ejecutan las instrucciones en un microprocesador? Bueno, ver cómo se ejecutan, no lo podemos ver, trabaja demasiado rápido para el ojo humano. Pero en la siguiente página puedes ver, pulsando en el botón run, una simulación de un procesador ejecutando instrucciones. Se trata de un antiguo MOS6502 de 8 bits (comparado con los actuales microprocesadores de 64 bits).



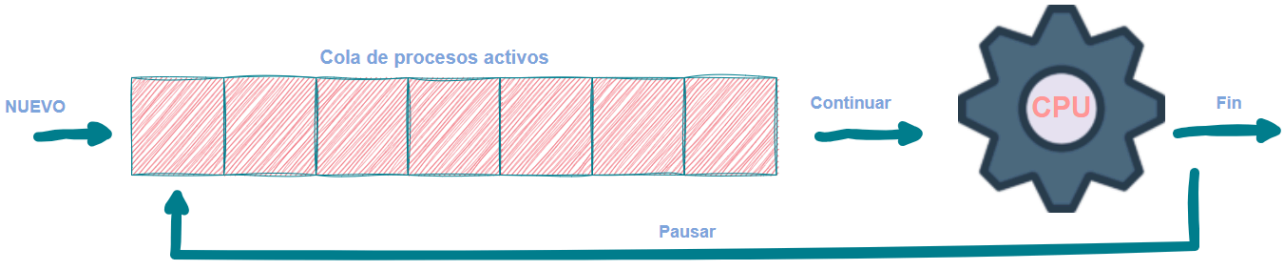
Ver una simulación de un procesador ejecutando instrucciones

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Estados de un Proceso

Si el sistema tiene que repartir el uso del microprocesador entre los distintos procesos, ¿qué le sucede a un proceso cuando no se está ejecutando? Y, si un proceso está esperando datos, ¿por qué el equipo hace otras cosas mientras que un proceso queda a la espera de datos?

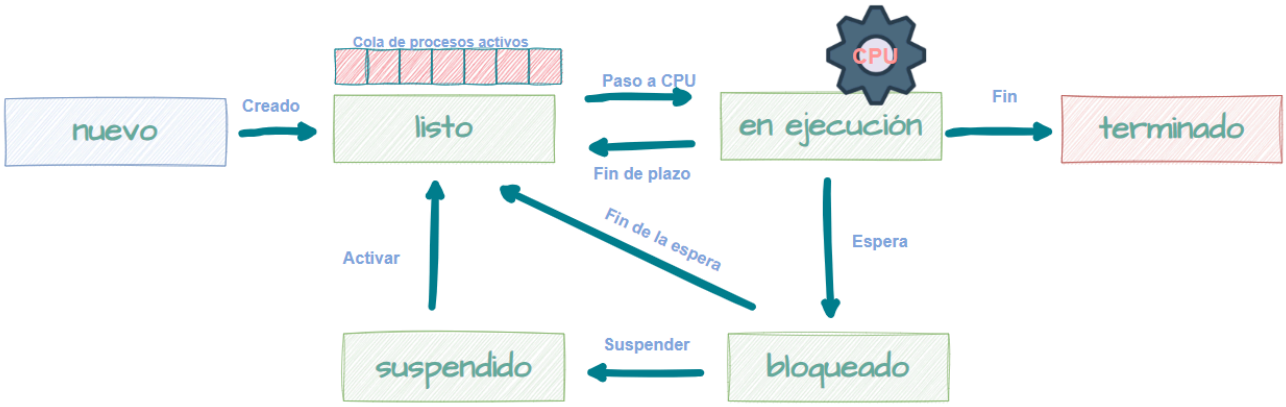
Veamos con detenimiento, cómo es que el SO controla la ejecución de los procesos. Ya comentamos en el apartado anterior, que el SO es el encargado de la gestión de procesos. En el siguiente gráfico, podemos ver un esquema muy simple de cómo podemos planificar la ejecución de varios procesos en una CPU.



En este esquema, podemos ver:

Los procesos nuevos, entran en la cola de procesos activos en el sistema.
Los procesos van avanzando posiciones en la cola de procesos activos, hasta que les toca el turno para que el SO les conceda el uso de la CPU.
El SO concede el uso de la CPU, a cada proceso durante un tiempo determinado y equitativo, que llamaremos quantum. Un proceso que consume su quantum, es pausado y enviado al final de la cola.
Si un proceso finaliza, sale del sistema de gestión de procesos.
Esta planificación que hemos descrito, resulta equitativa para todos los procesos (todos van a ir teniendo su quamtum de ejecución). Pero se nos olvidan algunas situaciones y características de nuestros los procesos:

Cuando un proceso, necesita datos de un archivo o una entrada de datos que deba suministrar el usuario; o, tiene que imprimir o grabar datos; cosa que llamamos 'el proceso está en una operación de entrada/salida' (E/S para abreviar). El proceso, queda bloqueado hasta que haya finalizado esa E/S. El proceso es bloqueado, porque, los dispositivos son mucho más lentos que la CPU, por lo que, mientras que uno de ellos está esperando una E/S, otros procesos pueden pasar a la CPU y ejecutar sus instrucciones. Cuando termina la E/S que tenga un proceso bloqueado, el SO, volverá a pasar al proceso a la cola de procesos activos, para que recoja los datos y continúe con su tarea (dentro de sus correspondientes turnos).
Sólo mencionar (o recordar), que cuando la memoria RAM del equipo está llena, algunos procesos deben pasar a disco (o almacenamiento secundario) para dejar espacio en RAM que permita la ejecución de otros procesos.



> Todo proceso en ejecución, tiene que estar cargado en la RAM física del equipo o memoria principal, así como todos los datos que necesite.

Hay procesos en el equipo cuya ejecución es crítica para el sistema, por lo que, no siempre pueden estar esperando a que les llegue su turno de ejecución, haciendo cola. Por ejemplo, el propio SO es un programa, y por lo tanto un proceso o un conjunto de procesos en ejecución. Se le da prioridad, evidentemente, a los procesos del SO, frente a los procesos de usuario.
Con todo lo anterior, podemos quedarnos con los siguientes estados en el ciclo de vida de un proceso:

- Nuevo. Proceso nuevo, creado.
- Listo. Proceso que está esperando la CPU para ejecutar sus instrucciones.
- En ejecución. Proceso que actualmente, está en turno de ejecución en la CPU.
- Bloqueado. Proceso que está a la espera de que finalice una E/S.
- Suspendido. Proceso que se ha llevado a la memoria virtual para liberar, un poco, la RAM del sistema.
- Terminado. Proceso que ha finalizado y ya no necesitará más la CPU.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

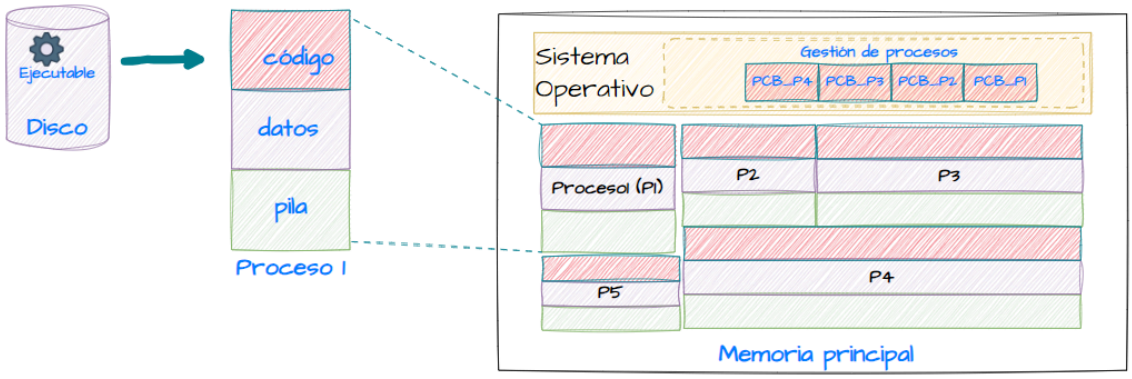
Planificación de procesos por el Sistema Operativo.

Entonces, ¿un proceso sabe cuando tiene o no la CPU? ¿Cómo se decide qué proceso debe ejecutarse en cada momento?

Hemos visto que un proceso, desde su creación hasta su fin (durante su vida), pasa por muchos estados. Esa transición de estados, es transparente para él, todo lo realiza el SO. Desde el punto de vista de un proceso, él siempre se está ejecutando en la CPU sin esperas. Dentro de la gestión de procesos vamos a destacar dos componentes del SO que llevan a cabo toda la tarea: el cargador y el planificador.

El cargador es el encargado de crear los procesos. Cuando se inicia un proceso (para cada proceso), el cargador, realiza las siguientes tareas:

1. Carga el proceso en memoria principal. Reserva un espacio en la RAM para el proceso. En ese espacio, copia las instrucciones del fichero ejecutable de la aplicación, las constantes y, deja un espacio para los datos (variables) y la pila (llamadas a funciones). Un proceso, durante su ejecución, no podrá hacer referencia a direcciones que se encuentren fuera de su espacio de memoria; si lo intentara, el SO lo detectará y generará una excepción (produciendo, por ejemplo, los típicos pantallazos azules de Windows).
2. Crea una estructura de información llamada PCB (Bloque de Control de Proceso). La información del PCB, es única para cada proceso y permite controlarlo. Esta información, también la utilizará el planificador. Entre otros datos, el PCB estará formado por:
    - Identificador del proceso o PID. Es un número único para cada proceso, como un DNI de proceso.
    - Estado actual del proceso: en ejecución, listo, bloqueado, suspendido, finalizando.
    - Espacio de direcciones de memoria donde comienza la zona de memoria reservada al proceso y su tamaño.
    - Información para la planificación: prioridad, quamtum, estadísticas, ...
    - Información para el cambio de contexto: valor de los registros de la CPU, entre ellos el contador de programa y el puntero a pila. Esta información es necesaria para poder cambiar de la ejecución de un proceso a otro.
    - Recursos utilizados. Ficheros abiertos, conexiones, …

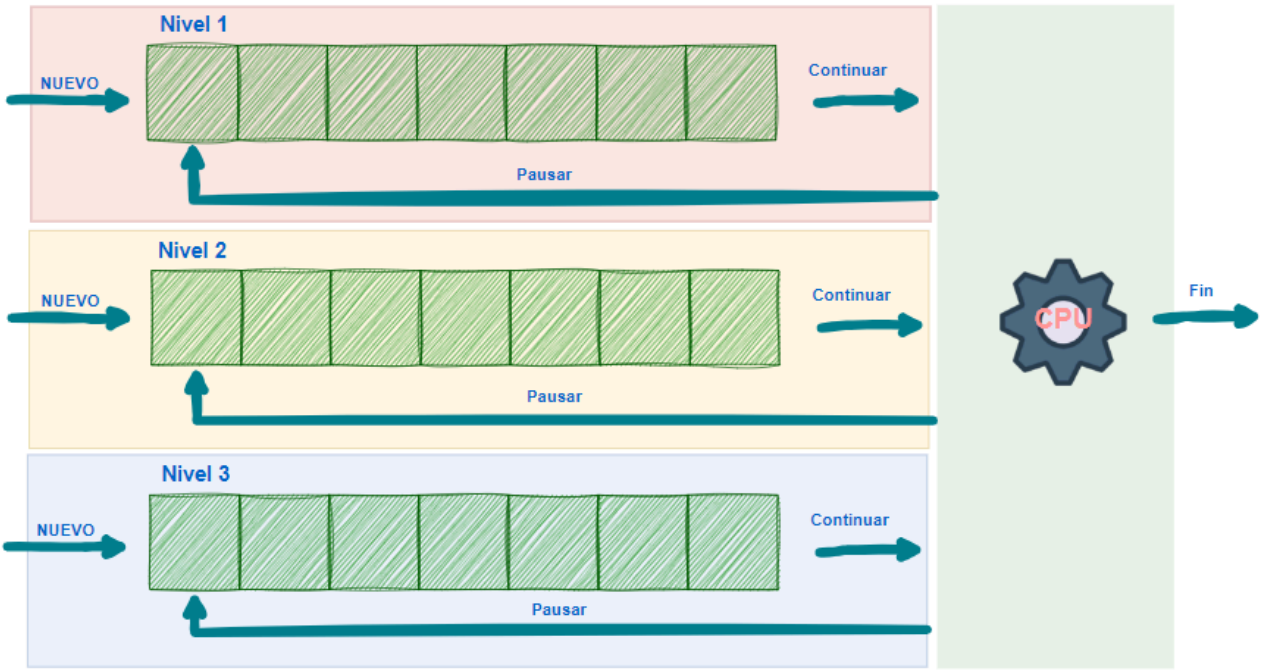


Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Planificación de procesos por el Sistema Operativo (II)

Una vez que el proceso ya está cargado en memoria, será el planificador el encargado de tomar las decisiones relacionadas con la ejecución de los procesos. Se encarga de decidir qué proceso se ejecuta y cuánto tiempo se ejecuta. El planificador es otro proceso que, en este caso, es parte del SO. La política en la toma de decisiones del planificador se denominan: algoritmo de planificación. Los más importantes son:

- **Round-Robin**. Este algoritmo de planificación favorece la ejecución de procesos interactivos. Es aquél en el que cada proceso puede ejecutar sus instrucciones en la CPU durante un quamtum. Si no le ha dado tiempo a finalizar en ese quamtum, se coloca al final de la cola de procesos listos, y espera a que vuelva su turno de procesamiento. Así, todos los procesos listos en el sistema van ejecutándose poco a poco.
- **Por prioridad**. En el caso de Round-Robin, todos los procesos son tratados por igual. Pero existen procesos importantes, que no deberían esperar a recibir su tiempo de procesamiento a que finalicen otros procesos de menor importancia. En este algoritmo, se asignan prioridades a los distintos procesos y la ejecución de estos, se hace de acuerdo a esa prioridad asignada. Por ejemplo: el propio planificador tiene mayor prioridad en ejecución que los procesos de usuario, ¿no crees?
- **Múltiples colas**. Es una combinación de los dos anteriores y el implementado en los sistemas operativos actuales. Todos los procesos de una misma prioridad, estarán en la misma cola. Cada cola será gestionada con el algoritmo Round-Robin. Los procesos de colas de inferior prioridad no pueden ejecutarse hasta que no se hayan vaciado las colas de procesos de mayor prioridad.



En la planificación (scheduling) de procesos se busca conciliar los siguientes objetivos:

- **Equidad**. Todos los procesos deben poder ejecutarse.
- **Eficacia**. Mantener ocupada la CPU un 100 % del tiempo.
- **Tiempo de respuesta**. Minimizar el tiempo de respuesta al usuario.
- **Tiempo de regreso**. Minimizar el tiempo que deben esperar los usuarios de procesos por lotes para obtener sus resultados.
- **Rendimiento**. Maximizar el número de tareas procesadas por hora.

En el siguiente enlace puedes ver una simulación del algoritmo de planificación Round-Robin, en él podrás ver cómo los procesos van tomando su turno de ejecución en la CPU hasta su finalización.



Enlace a un simulador del algoritmo de planificación Round-Robin.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Cambio de contexto en la CPU.

> **Una CPU ejecuta instrucciones, independientemente del proceso al que pertenezcan. Entonces, ¿cómo consigue unas veces ejecutar las instrucciones de un proceso y luego de otro y otro y otro..., sin que se mezclen los datos de unos con otros?**

Un proceso es una unidad de trabajo completa. El sistema operativo es el encargado de gestionar los procesos en ejecución de forma eficiente, intentando evitar que haya conflictos en el uso que hacen de los distintos recursos del sistema. Para realizar esta tarea de forma correcta, se asocia a cada proceso un conjunto de información (PCB) y de unos mecanismos de protección (un espacio de direcciones de memoria del que no se puede salir y una prioridad de ejecuión).

Imaginemos que, en nuestro equipo, en un momento determinado, podemos estar escuchando música, editando un documento, al mismo tiempo, chateando con otras personas y navegando en Internet. En este caso, tendremos ejecutándose en el sistema cuatro aplicaciones distintas, que pueden ser: el reproductor multimedia VLC, el editor de textos writer de OpenOffice, el Messenger y el navegador Firefox. Todos ellos, ejecutados sin fallos y cada uno haciendo uso de sus datos.

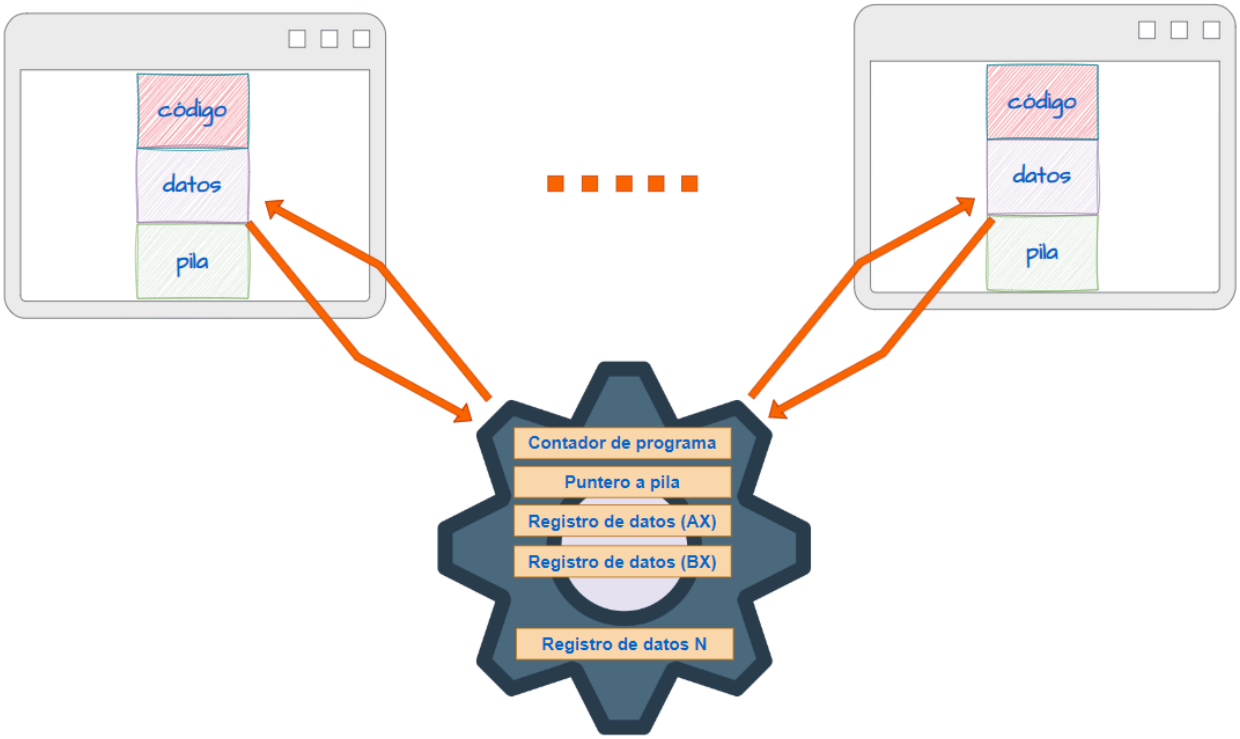
El sistema operativo (el planificador), al realizar el cambio una aplicación a otra, tiene que guardar el estado en el que se encuentra el microprocesador y cargar el estado en el que estaba el microprocesador cuando cortó la ejecución de otro proceso, para continuar con ese. Pero, ¿qué es el estado de la CPU?

Una CPU, además de circuitos encargados de realizar las operaciones con los datos (llamados circuitos operacionales), tiene unas pequeños espacios de memoria (llamados registros), en los que se almacenan temporalmente la información que, en cada instante, necesita la instrucción que esté procesando la CPU. El conjunto de registros de la CPU es su estado.

Entre los registros, destacamos el Registro Contador de Programa y el puntero a la pila.

- El **Contador de Programa**, en cada instante almacena la dirección de la siguiente instrucción a ejecutar. Recordemos, que cada instrucción a ejecutar, junto con los datos que necesite, es llevada desde la memoria principal a un registro de la CPU para que sea procesada; y, el resultado de la ejecución, dependiendo del caso, se vuelve a llevar a memoria (a la dirección que ocupe la correspondiente variable). Pues el Contador de Programa, apunta a la dirección de la siguiente instrucción que habrá que traer de la memoria, cuando se termine de procesar la instrucción en curso. Este Contador de Programa nos permitirá continuar en cada proceso por la instrucción en dónde lo hubiéramos dejado todo.
- El **Puntero a Pila**, en cada instante apunta a la parte superior de la pila del proceso en ejecución. En la pila de cada proceso es donde será almacenado el contexto de la CPU. Y de donde se recuperará cuando ese proceso vuelva a ejecutarse.

La CPU realiza un cambio de contexto cada vez que cambia la ejecución de un proceso a otro distinto. En un cambio de contexto, hay que guardar el estado actual de la CPU y restaurar el estado de CPU del proceso que va a pasar a ejecutar.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Servicios. Hilos.

> La conclusión que estamos sacando, es, que todo lo que se ejecuta en un equipo es un programa y que, cuando está en ejecución, se llama proceso. Entones ¿qué son los servicios? ¿y los hilos?

En este apartado, haremos una breve introducción a los conceptos servicio e hilo, ya que los trataremos en profundidad en el resto de unidades de este módulo.

El ejemplo más claro de hilo o thread, es un juego. El juego, es la aplicación y, mientras que nosotros controlamos uno de los personajes, los 'malos' también se mueven, interactúan por el escenario y quitan vida. Cada uno de los personajes del juego es controlado por un hilo. Todos los hilos forman parte de la misma aplicación, cada uno actúa siguiendo un patrón de comportamiento. El comportamiento es el algoritmo que cada uno de ellos seguirá. Sin embargo, todos esos hilos comparten la información de la aplicación: el número de vidas restantes, la puntuación obtenida hasta eso momento, la posición en la que se encuentra el personaje del usuario y el resto de personajes, si ha llegado el final del juego, etc. Como sabemos, esas informaciones son variables. Pues bien, un proceso, no puede acceder directamente a la información de otro proceso. Pero, los hilos de un mismo proceso están dentro de él, por lo que comparten la información de las variables de ese proceso.

Realizar cambios de contexto entre hilos de un mismo proceso, es más rápido y menos costoso que el cambio de contexto entre procesos, ya que sólo hay que cambiar el valor del registro contador de programa de la CPU y no todos los valores de los registros de la CPU.

> Un proceso, estará formado por, al menos, un hilo de ejecución.
> Un proceso es una unidad pesada de ejecución. Si el proceso tiene varios hilos, cada hilo, es una unidad de ejecución ligera.

Para saber más

¿Sabes lo que es Hyper-Threading (HT)? Es una tecnología patentada por Intel, que incorporó en sus micros Pentium4 de un sólo núcleo para que el propio micro (hardware) simulara la existencia de 2 núcleos lógicos, para obtener mayor productividad en procesos de más de un hilo, ya que cada núcleo lógico gestionará cada hilo de forma casi independiente. Esta tecnología la eliminó en sus Core 2 Duo y Quad; ya que al existir más de un núcleo hardware no hacía falta simular la existencia de más de un núcleo por núcleo físico. Y lo ha vuelto a introducir en su familia de microprocesadores i7, i5 e i3. Estos últimos, por

cada núcleo físico, simulan 2 núcleos lógicos. Buscan así incrementar la productividad del micro.

Enlace en wikipedia.org a su entrada sobre Hyper-Threading.

Un servicio es un proceso que, normalmente, es cargado durante el arranque del sistema operativo. Recibe el nombre de servicio, ya que es un proceso que queda a la espera de que otro le pida que realice una tarea. Por ejemplo, tenemos el servicio de impresión con su típica cola de trabajos a imprimir. Nuestra impresora imprime todo lo que recibe del sistema, pero se debe tener cuidado, ya que si no se le envían los datos de una forma ordenada, la impresora puede mezclar las partes de un trabajo con las de otro, incluso dentro del mismo folio. El servicio de impresión, es el encargado de ir enviando los datos de forma correcta a la impresora para que el resultado sea el esperado. Además, las impresoras, no siempre tienen suficiente memoria para guardar todos los datos de impresión de un trabajo completo, por lo que el servicio de impresión se los dará conforme vaya necesitándolos. Cuando finalice cada trabajo, puede notificárselo al usuario. Si en la cola de impresión, no hay trabajos pendientes, el servicio de impresión quedará a la espera y podrá avisar a la impresora para que quede en StandBy.

Como este, hay muchos servicios activos o en ejecución en el sistema, y no todos son servicios del sistema operativo, también hay servicios de aplicación, instalados por el usuario y que pueden lanzarse al arrancar el sistema operativo o no, dependiendo de su configuración o cómo los configuremos.

> Un servicio, es un proceso que queda a la espera de que otros le pida que realice una tarea.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Creación de procesos.

> Cada instancia de una aplicación en ejecución es un proceso. Cada proceso dispone de un conjunto de instrucciones, un estado de los registros del procesador, un espacio de memoria y un estado en lo referente a la gestión que hace de él el planificador del sistema operativo. ¿Tiene sentido, por lo tanto, hablar de programación de aplicaciones multiprocesos si una aplicación cuando se ejecuta constituye uno único?
> La respuesta es sí, siempre y cuando se acote el significado del concepto "aplicación multiproceso", ya que se trata un término cuyo alcance es difuso. En esta unidad se aborda la programación de aplicaciones multiproceso como la capacidad de coordinar la ejecución de un conjunto de aplicaciones para lograr un objetivo común.
> Por ejemplo, si se dispone de un sistema compuesto por un conjunto de procesos que deben ejecutarse de manera individual, pero que tienen dependencias entre sí, se necesita disponer de un mecanismo de gestión y coordinación.
> Las necesidades que se deben satisfacer para poder programar un sistema basado en la ejecución de múltiples procesos son las siguientes:
>
> - Poder arrancar un proceso y hacerle llegar los parámetros de ejecución.
> - Poder quedar a la espera de que el proceso termine.
> - Poder recoger el código de finalización de ejecución para determinar si el proceso se ha ejecutado correctamente o no.
> - Poder leer los datos generados por el proceso para su tratamiento.

En muchos casos necesitaremos que una aplicación lance varios procesos. Esos procesos pueden realizar cada uno una tarea distinta o todos la misma. Por ejemplo, imaginemos un editor de texto plano sencillo. Estamos acostumbrados a que los distintos ficheros abiertos se muestren en pestañas independientes, pero ¿cómo implementamos eso?

Las clases que vamos a necesitar para la creación de procesos, son:

- Clase **java.lang.Process**. Proporciona los objetos Proceso, por los que podremos controlar los procesos creados desde nuestro código.
- Clase **java.lang.Runtime**. Clase que permite lanzar la ejecución de un programa en el sistema. Sobre todos son interesantes los métodos exec() de esta clase, por ejemplo:Runtime.exec(String comando); devuelve un objeto Process que representa al proceso en ejecución que está realizando la tarea comando.
  La ejecución del método exec() puede lanzar las excepciones: SecurityException, si hay administración de seguridad y no tenemos permitido crear subprocesos. IOException, si ocurre un error de E/S. NullPointerException y IllegalArgumentException, si commando es una cadena nula o vacía.

Veamos un ejemplo sencillo. Si queremos editar varios ficheros de texto a la vez, necesitamos que sean creados tantos procesos del editor de texto, como documentos queramos editar. Vamos a utilizar la aplicación de ejemplo 'Editor de texto (Document Editor)', para que nos permita editar varios documentos al mismo tiempo en distintas instancias del editor.





La siguiente clase escrita en Java representa un proceso que, tras intentar realizar para lo que se está programado, devuelve un código de retorno 103 utilizando el método do exit de la clase System, que significa que ha ocurrido un error de un tipo determinado.

Código Java

```
  public class ProcesoSecundario {
        public static void main(String[] args) {
                System.err.println("Proceso secundario...");
                System.exit(103);
        }
}
```

Por otra parte, la clase siguiente representa un proceso que ejecuta el proceso anterior (método exec), queda a la espera de que termine (método waitFor), recoge el valor de finalización del proceso y lo evalúa para tomar una decisión.

Código Java

```
  public class ProcesoPrincipal {
        public static void main(String[] args) {
                try {
                        String [] infoProceso = {"java",
"unidades.unidad1.ejemplos.ProcesoSecundario"};
                        Process proceso  =
Runtime.getRuntime().exec(infoProceso);
                        int valorRetorno =  proceso.waitFor();
                        if(valorRetorno == 0) {
                                System.out.println("El proceso se ha
completado satisfactoriamente");
                        }else {
                                System.out.println("El proceso ha fallado.
Código error:"+ valorRetorno);
                        }
                }catch(Exception e) {
                        e.printStackTrace();
                }
        }
}
```

El proceso principal recoge el código de finalización del proceso secundario, lo evalúa y en función del valor que toma elige uno u otro camino.

En este ejemplo se muestra uno y otro mensaje de error de texto por la salida estándar en función del código de finalización del proceso secundario. Si en lugar de mostrar mensajes se hubiesen ejecutado procesos, se habría creado un flujo de ejecución de trabajos.

Este tipo de sincronización se puede realizar entre procesos escritos en distintos lenguajes de programación

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Creación de procesos en Java

En Java, la creación de un proceso puede realizarse de dos maneras diferentes:

- Utilizando la clase java.lang.Runtime.
- Utilizando la clase java.lang.ProcessBuilder.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Creación de procesos con Runtime

Toda aplicación Java tiene una única instancia de la clase Runtime que permite que la propia aplicación interactúe con su entorno de ejecución a través del método estático getRuntime. Este método proporciona un "canal" de comunicación entre la aplicación y su entorno, posibilitando la interacción con el sistema operativo a través del método exec.

El siguiente código Java genera un proceso en Windows indicando al entorno de ejecución (al sistema operativo) que ejecute el bloc de notas a través del programa "Notepad.exe". En este caso, la llamada se realiza sin parámetros y sin gestionar de ninguna manera el proceso generado.

```
Runtime.getRuntime().exec("Notepad.exe");
```

En muchos casos los procesos necesitan parámetros para iniciarse. El método exec puede recibir una cadena de caracteres (un objeto de la clase String) y en dicha cadena, separados por espacios, se indicarán, además del programa que se va a ejecutar, los diferentes parámetros.

En el siguiente código se está ejecutando el bloc de notas indicando que "notas.txt" es el fichero que debe abrir o crear si no existe.

```
Runtime.getRuntime().exec("Notepad.exe notas.txt");
```

Alternativamente se puede crear el proceso proporcionando un array de objetos String con el nombre del programa y los parámetros

```
String[] infoProceso = {"Notepad.exe","notas.txt"};

Runtime.getRuntime().exec(infoProceso);
```

Si se necesita esperar a que el proceso ejecutado termine y conocer el estado en que ha finalizado dicha ejecución, se puede utilizar el método waitFor. Este método suspende la ejecución del programa que ha arrancado el proceso quedando a la espera de que este termine, proporcionando además el código de finalización.

```
String[] infoProceso = {"Notepad.exe","notas.txt"};

Process proceso = Runtime.getRuntime().exec(infoProceso);

int codigoRetorno = proceso.waitFor();

System.out.println("Fin de la ejecución:"+ codigoRetorno);
```

La clase Process representa al proceso de ejecución y permite obtener información sobre éste. Los principales métodos que proporciona dicha clase son los que se recogen en la siguiente tabla:

Método	Descripción
`destroy()`	Destruye el proceso sobre el que se ejecuta.
`exitValue()`	Devuelve el valor de retorno del proceso cuando este finaliza. Sirve para controlar el estado de la ejecución.
`getErrorStream()`	Proporciona un `InputStream` conectado a la salida de error del proceso.
`getInputStream()`	Proporciona un `InputStream` conectado a la salida normal del proceso.
`getOutputStream()`	Proporciona un `OutputStream` conectado a la entrada normal del proceso.
`isAlive()`	Determina si el proceso está o no en ejecución.
`waitFor()`	Detiene la ejecución del programa que lanza el proceso a la espera de que este último termine.

En el siguiente enlace podrás acceder a los ejemplos completos que usan el código anterior:



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Creación de procesos con ProcessBuilder

La clase ProcessBuilder permite, al igual que Runtime, crear procesos.

La creación más sencilla de un proceso se realiza con un único parámetro en el que se indica el programa a ejecutar. Es importante saber que esta construcción no supone la ejecución del proceso.

```
new ProcessBuilder("Notepad.exe");
```

La ejecución del proceso se realiza a partir de la invocación al método start:

```
new ProcessBuilder("Notepad.exe").start();
```

El constructor de ProcessBuilder admite parámetros que serán entregados al proceso que se crea

```
new ProcessBuilder("Notepad.exe","datos.txt").start();
```

Al igual que ocurre con el método exec de la clase Runtime, el método start de ProcessBuilder proporciona un proceso como retorno, lo que posibilita la sincronización y gestión de éste.

```
Process proceso =  new ProcessBuilder("Notepad.exe","datos.txt").start();

int valorRetorno = proceso.waitFor();

System.out.println("Valor retorno:"+ valorRetorno);
```

El metodo start permite crear múltiples subprocesos a partir de una única instancia de ProcessBuilder. El siguiente código crea diez instancias de bloc de notas de Windows.

```
ProcessBuilder pBuilder = new ProcessBuilder("Notepad.exe");

for(int i=0;i<10;i++){

pBuilder.start();

}
```

Además del método start, la clase ProcessBuilder dispone de métodos para consultar y gestionar algunos parámetros relativos a la ejecución del proceso. Los métodos más relevantes de ProcessBuilder se muestran en la siguiente tabla:

Método	Descripción
`start()`	Inicia un nuevo proceso usando los atributos especificados.

`command()`	Permite obtener o asignar el programa y los argumentos de la instancia de `ProcessBuilder`.
`directory()`	Permite obtener o asignar el directorio de trabajo del proceso.
`environment()`	Proporciona información sobre el entorno de ejecución del proceso.
`redirectError()`	Permite determinar el destino de la salida de errores.
`redirectInput()`	Permite determinar el origen de la entrada estándar.
`redirectOutput()`	Permite determinar el destino de la salida estándar.

```
ProcessBuilder pBuilder = new ProcessBuilder("Notepad.exe","datos.txt");

pBuilder.directory(new File("c:/directorio_salida"));
```

Para acceder a la información del entorno de ejecución, el método environment devuelve un objeto Map con la información proporcionada por el sistema operativo. El siguiente ejemplo muestra por pantalla el número de procesadores disponibles en el sistema:

```
ProcessBuilder pBuilder = new ProcessBuilder("Notepad.exe","datos.txt");

java.util.Map env = pBuilder.environment();

System.out.println("Número de procesadores:"+ env.get("NÚMERO DE PROCESADO
```

En el siguiente enlace accederás al código completo de creación de proceso



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Ejemplo ejecutar shell script

Código: crearUsuario.sh

```
#!/bin/bash


#sudo useradd -m -s /bin/bash $1

sudo adduser --disabled-password --gecos "" --shell /bin/bash $1

sudo echo $1:$2 | chpasswd

#pass_usuario_pass_encrypted=openssl passwd -6 -salt gromenauer $2

#useradd -m -s /bin/bash -p (echo $2 | openssl passwd -6 -salt gromenauer

#useradd -m -s /bin/bash -p $pass_usuario_pass_encrypted $1


#Ocultar la carpeta del usuario a los demás

sudo chmod 770 "/home/$1"

# Se muestra la información del usuario creado

#echo "El usuario $1 ha sido creado con la contraseña:" $(echo $2 | opens

echo "El usuario $ 1 ha sido creado con la contraseña:" $pass_usuario_pas
```

Comando para obtener la contraseña encriptada

```
sudo openssl passwd -5 -salt gromenauer contraseña

Ejemplo:

La contraseña es admin
```

```
sudo openssl passwd -5 -salt gromenauer admin

Devuelve:

$6$gromenauer$GpYY6CAKv3uexUtI3RbOE/smJ6zpBmCcSg9tnyUPHXwbkVX0eg2o76JokaF
```

Código Java

```
import java.io.BufferedReader;

import java.io.IOException;

import java.io.InputStream;

import java.io.InputStreamReader;

public class EjecutarProceso {

    public static void main(String[] args) {

        try {

            String comando = "/home/mint/Escritorio/crearUsuario.sh";
            ProcessBuilder pb = new ProcessBuilder(comando, "usu13",
"$6$gromenauer$GpYY6CAKv3uexUtI3RbOE/smJ6zpBmCcSg9tnyUPHXwbkVX0eg2o76Joka
            pb.redirectErrorStream(true);
            Process proceso = pb.start();
            int codigoSalida = proceso.waitFor();
            System.out.println("El script ha sido ejecutado. Código de
salida: " + codigoSalida);
            InputStream is = proceso.getInputStream();
            BufferedReader br = new BufferedReader(new
InputStreamReader(is));
            String linea;
            while((linea = br.readLine())!=null) {
                 System.out.println(linea);
            }
            br.close();
        } catch (IOException e) {
            e.printStackTrace();
        } catch (InterruptedException e) {
```

```
            e.printStackTrace();

        }
    }
}
```

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Crear procesos en Python

La forma más sencilla de lanzar un proceso sería usar el método estático run de un subprocess, al cual de forma básica podemos pasarle:

- La ruta completa de un fichero ejecutable (.exe).
- El nombre de un ejecutable que pueda localizarse a través del path del sistema y que puede completarse con la extensión .exe en caso de no indicarla.
- El ejecutable con su lista de parámetros, con lo cual podríamos, por ejemplo lanzar un editor y pasarle el fichero a ser abierto.

En la actualidad, se pueden crear procesos indistintamente con "call" o "run", si bien a partir de la versión 3.5 se recomienda el uso de "run" por la posibilidad de que "call" pase a estar deprecated ("obsoleto") en futuras versiones.

Código Python

```
 import subprocess
try:
  subprocess.run(['Notepad.exe',])
  subprocess.run(['c:/windows/notepad.exe',])
  subprocess.run(['Notepad.exe','texto.txt'])
except subprocess.CalledProcessError as e:
    print(e.output)
```

En este código ya que tener en cuenta que, en caso de no poder crear un proceso, por ejemplo, de no existir el fichero o programa asociado, retornaría un valor null y daría una excepción que fácilmente se puede capturar en un try...catch.

Ejercicio propuesto

Lanzar un proceso que abra el navegador con la búsqueda de Google de un término solicitado al usuario.

Si queremos enviar parámetros al proceso para ser ejecutado, éstos deberán ser enumerados en la lista, de forma posterior al nombre de proceso. En el código siguiente, podemos ver cómo hacer cinco pings consecutivos, enviando los argumentos correspondientes, para comprobar la conectividad con un URL dado.

Para aprender más...

Toda la documentación sobre subprocesos en Python se puede consultar en



Código Python

```
import subprocess

try:
  subprocess.run(["ping", "www.educarex.es","-n","5"])
  #en linux
  #subprocess.run(["ping", "www.educarex.es","-c","5"])
except subprocess.CalledProcessError as e:
    print(e.output)
```

Si antes de ejecutar un proceso queremos establecer los parámetros de ejecución, debemos proporcionar información a través de la clase STARTINFO, la cual solo está disponible para Windows. Las propiedades más destacables son las siguientes:

- dwFlags: es un bit que determina el uso de ciertos atributos, si el proceso crea una ventana.
- hStolnput: determina el gestor de entrada del proceso en caso de activar el dwFlags en STARTF_USESTDHANDLES. En caso de no activarlo, se toma el búfer de teclado.
- hStdOutput: de igual manera que la anterior propiedad, determina el gestor de salida. En caso de no activarlo, se toma el búfer de ventana.
- hStdError: de igual manera para el gestor de error estándar del proceso. En caso de no activarlo, se toma el búfer de ventana.
- wShowWindow: si dwFlags especifica el valor STARTF_USESTDHANDLES, este atributo toma los valores del parámetro nCmdShow para la función ShowWin-dow: por ejemplo, el valor 3 para crear la ventana en formato maximizad.
- IpAttributeList: con un diccionario de atributos adicionales para la creación del proceso.

En el siguiente código podemos ver de qué manera lanzar la ejecución del bloc de notas sobre una información de inicio previa, como la ventana maximizada.

Código Python

```
import subprocess
```

```
def iniciaPrograma():
  try:
    SW_SHOWMAXIMIZED = 3
    info = subprocess.STARTUPINFO()
    info.dwFlags |= subprocess.STARTF_USESHOWWINDOW
    info.wShowWindow = SW_SHOWMAXIMIZED
    subprocess.Popen('Notepad.exe', startupinfo=info)
  except subprocess.CalledProcessError as e:
    print(e.output)
iniciaPrograma()
input("Pulsa una tecla para terminar la ejecución")
```

Ejercicio propuesto

Observar el administrador de tareas (Windows) o el monitor de sistema Linux e identificar algún proceso por su PID. Elaborar un programa que acceda a él y muestre información como su nombre, lo elimine, etc.

Una forma alternativa para consultar los procesos del sistema en Windows podría ser con la invocación al proceso (comando) **process**, a través del **wmic**, tal y como se muestra en el siguiente código.

Código Python

```
import subprocess

# obtención de los procesos
Datos = subprocess.check_output(['wmic', 'process', 'list', 'brief'])
a = str(Datos)
try:
    for i in range(len(a)):
        print(a.split("\\r\\r\\n")[i])
except IndexError as e:
    print("Finalizado")
```

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Acceso

En el apartado superior hemos creado y puesto en ejecución procesos, pero, si posteriormente necesitamos acceder a ello para consultar o cambiar sus propiedades o estado, debemos guardar una referencia, por lo que tenemos que guardar el retorno de un call(), run() o Popen().

Código Python

```
import subprocess
import time



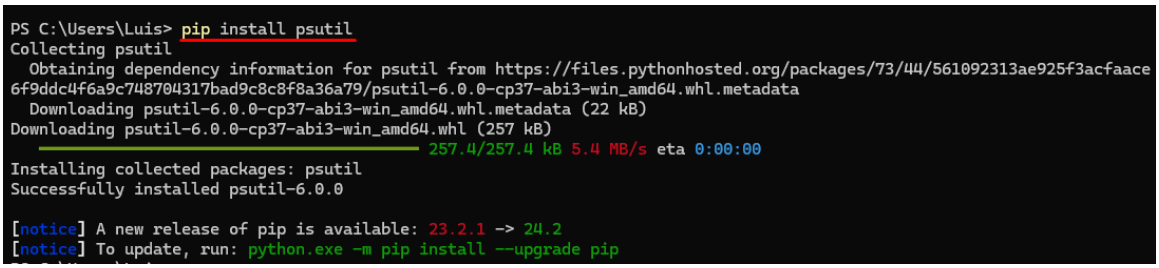
def CrearProceso():
  try:
    SW_SHOWMAXIMIZED = 3
    info = subprocess.STARTUPINFO()
    info.dwFlags |= subprocess.STARTF_USESHOWWINDOW
    info.wShowWindow = SW_SHOWMAXIMIZED
    proc = subprocess.Popen('notepad.exe', startupinfo=info)
    return proc
  except subprocess.CalledProcessError as e:
    print(e.output)
p = CrearProceso()
print ("El PID de este proceso es: " + str(p.pid))
time.sleep(5)
```

Si lo que queremos es acceder a los procesos del sistema, podemos recuperarlos a través de la librería psutil, y la iteración a través de la función process_iter(), como podemos ver en el código

Importante

La librería psutil no viene instalada como opción predeterminada: por lo tanto, es necesario instalarla, lo cual se puede hacer en consola con el comando

pip install psutil



Código Python

```
#instalar psutil: pip install psutil
import psutil

try:
  for proc in psutil.process_iter():
    #Obtener el nombre y el PID de cada proceso
    processName = proc.name()
    processID = proc.pid
    print(processName , ' ::: ', processID)
#except (psutil.NoSuchProcess, psutil.AccessDenied,
psutil.ZombieProcess) as e:
except:
  print("error")
```

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Eliminación y espera

Un proceso puede terminar de forma natural después de finalizar su ejecución.
También podemos finalizar un proceso creado o cualquier otro proceso del sistema. Cabe recordar aquí que, como usuario programador, estamos limitados por los permisos que establezca el sistema y que, cuando se ejecute nuestro programa, lo hará bajo los privilegios del usuario que lo inicie. Por este motivo, no es posible eliminar un proceso sin el beneplácito del sistema operativo.
El método usado para eliminar un proceso es kill(), el cual en unix envía la señal SIGKILL y en Windows invoca la función terminate).
La eliminación de un proceso es asíncrona, lo cual significa que, si invocamos a kill() en una línea de código, al invocar la siguiente pudiera ser que el proceso todavía siga existiendo. Podemos entender esto con todo lo aprendido en los apartados previos acerca del cambio de contexto, las interrupciones, etc. Para saber si un proceso ha finalizado de forma efectiva podemos usar el método poll(), el cual nos devuelve el valor None si el proceso de terminación se ha completado.
Ver el siguiente código:

Código Python

```
import psutil

for proc in psutil.process_iter():
  try:
    # Obtener el nombre del proceso
    nombreProceso = proc.name()
    if proc.name() == "notepad.exe":
      PID = proc.pid
      print("Eliminando el proceso: ", nombreProceso , ' ::: ', PID)
      proc.kill()
  except (psutil.NoSuchProcess, psutil.AccessDenied,
psutil.ZombieProcess):
    print ("error")
```

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Ejecución asíncrona

En los ejemplos vistos hasta ahora, cada vez que invocabamos la ejecución de un proceso, nuestro programa quedaba paralizado a la espera de la finalización de dicho proceso. Si el proceso tenía una terminación definida, como puede ser la repetición de un número determinado de veces del comando ping, se puede observar una secuencia con cierta prontitud o continuidad. Sin embargo, cuando lanzamos el bloc de notas, delegamos el manejo y la finalización de este al usua rio, sin tener definido el tiempo de uso y, por lo tanto, el momento en el que el control volverá a nuestro programa. Estamos trabajando bajo las condiciones de la programación sincrona; es decir, la ejecución de una línea de código no empieza hasta que no haya concluido completamente la anterior. Esta es la manera más común de programar y, bajo estas condiciones, cuando llamamos a un proceso, con el método run(), por ejemplo, la ejecución de nuestro programa permanece bloqueada hasta que ese proceso no finaliza. Se dice, por lo tanto, que esta es una línea bloqueante.
En el extremo contrario está la programación asíncrona, en la que podemos hacer que algunas líneas puedan tener una ejecución independiente y no sea necesario esperar a su finalización para comenzar a ejecutar la siguiente. Aunque para el programador, establecer una ejecución asíncrona es algo conceptualmente sim-ple, en el fondo toda ejecución asíncrona conlleva una creación y sincronización automática de hilos, lo cual se verá en la siguiente unidad. Aunque existen más formas, el modo más sencillo de establecer el asincronismo se basa en los siguientes elementos:

- async: definición de una función que tiene alguna operación asíncrona en su interior.
- await: indicación de que no vamos a esperar por la finalización de una función, pues contiene operaciones asíncronas.
- asyncio.run(): método de la librería asyncio, que inicia la ejecución de una función asíncrona.

> **Código Python**
>
> ```
> from os import system
> import subprocess
> import asyncio
>
> def showNotepad1():
>   try:
>     subprocess.run(['Notepad.exe',])
>   except subprocess.CalledProcessError as e:
>       print(e.output)
> async def showNotepad2():
>   try:
>     await asyncio.create_subprocess_exec('notepad.exe')
>   except subprocess.CalledProcessError as e:
> ```

```
      print(e.output)

async def main():
  print ("inicio llamada síncrona")
  showNotepad1()
  print ("fin llamada síncrona")
  print ("inicio llamada asíncrona")
  await showNotepad2()
  print ("fin llamada asíncrona")
  print("pulsa una tecla para terminar")
  system ('Pause')

asyncio.run(main())
```

En el código anterior, podemos comparar la ejecución de dos procesos consecutivos del bloc de notas, intercalados entre mensajes para observar la diferencia entre ambos. La primera vez que se llama al bloc de notas se hace de manera asíncrona y podemos observar que el mensaje "fin de llamada síncrona" no aparece hasta que hayamos finalizado la ejecución del bloc. La segunda llamada es asíncrona await showNotepad2(), y esto lo podemos hacer porque dentro de la función usamos un método de ejecución de procesos que es de naturaleza asíncrona: asyncio.create_subprocess_exec).
En la llamada asíncrona podemos observar que las líneas de código siguen ejecutándose, y podemos ver el mensaje "fin de llamada síncrona" , pese a que el bloc de notas sigue activo. De alguna manera, podemos decir que corre en paralelo al resto del programa.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Comunicación entre procesos

Los sistemas operativos implementan varias técnicas de IPC, entre las que se en. cuentran las PIPE o tuberías, espacios compartidos de memoria, rutinas de in. tercambio, semáforos, monitores, etc. Incluso hay métodos para comunicar procesos que se ejecutan en ordenadores diferentes tales como PC, CORBA, MP1, sockets, etc. Muchas técnicas tienen un protocolo propietario muy particular lo cual hace que sea difícil aproximarse en detalle a todas ellas.

> **IPC**. **Inter-process communication**. Mecanismo que permite a los procesos comunicarse y sincronizarse entre sí.

Una forma muy común es capturar la entrada y/o salida de un proceso, lo cual tiene un aprovechamiento casi en exclusiva de los programas de consola, y más limitada para las que usan una interfaz gráfica. En estos casos, se puede capturar y redirigir las salidas estándar de entrada "stdin", salida "stalout" y error "stder.

> En https://test.rebex.net/ tenemos una colección de servidores de ensayo (ftp, ssh, imap, time...) con sus credenciales de usuario y contraseña. Con ellos podemos probar y verificar nuestros clientes.

En el siguiente código podemos observar cómo lanzar un cliente de ftp con una suce. sión de comandos que van desde la conexión con el usuario y contraseña, hasta el listado, hasta concluir con la descarga de un fichero. El proceso "ftp" se abre en el Shell del sistema ("cmd") mediante Popen. Usamos la entrada stoin asociada al PIPE de subproceso p1, para escribir todos los comandos.
Con el método communicatel) hacemos efectivo el envío de los comandos, con un tiempo límite de espera de cinco segundos. Este método nos devuelve una lista con las salidas stdout y stderr, de las cuales tan solo nos quedamos con la primera (de ahí el índice cero [O]).
Finalmente, y de forma secuencial, escribimos todas las respuestas stdout. Nótese que se ha descodificado a "cp850" para que se muestren § y tildes en el Windows instalado para hacer las pruebas; si bien esta codificación pudiera variar dependiendo del uso de códigos del sistema, por ejemplo, a "utf-8".

> **Código Python**

```
import subprocess

p1 = subprocess.Popen('ftp', shell=True, stdin=subprocess.PIPE,
stdout=subprocess.PIPE, stderr=subprocess.PIPE)
comandos = [b"verbose\n",
            b"open test.rebex.net\n",
            b"demo\n",
```

```
            b"password\n",
            b"ls\n",
            b"get readme.txt\n"]

for cmd in comandos:
  p1.stdin.write (cmd)

respuesta = p1.communicate(timeout=5)[0]
print (respuesta.decode("cp850", "ignore"))
```

Otro ejemplo de comunicación o intercambio de datos podría ser a través de una librería que internamente codifica y encapsula las correspondientes llamadas a la API del portapapeles de Windows. Dado que la librería no viene instalada inicial-mente, hay que hacerlo mediante el comando "pip install pywin32".
En el siguiente código, se muestra cómo enviar, vaciar el portapapeles y establecer un texto, para a continuación recuperarlo.

Código Python

```
#instalar previamente: "pip install pywin32"
import win32clipboard

#enviar datos al portapapeles
win32clipboard.OpenClipboard()
win32clipboard.EmptyClipboard()
win32clipboard.SetClipboardText("CICLOS-1DAM")
win32clipboard.CloseClipboard()

# obtener datos del portapapeles
win32clipboard.OpenClipboard()
datos = win32clipboard.GetClipboardData()
win32clipboard.CloseClipboard()
print (datos)
```

Hoy día, la técnica más extendida para el intercambio de información entre procesos es el uso de sockets. A ello encaminaremos nuestros esfuerzos en próximas unidades en cuanto entendamos los hilos, que es el siguiente grado en el fraccionamiento de procesos.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Comandos para la gestión de procesos

Es cierto que podemos pensar que ya no necesitamos comandos. Y que podemos desterrar el intérprete de comandos, terminal o shell. Hay múltiples motivos por los que esto no es así:

- En el apartado anterior, hemos visto que necesitamos comandos para lanzar procesos en el sistema.
- Además de las llamadas al sistema, los comandos son una forma directa de pedirle al sistema operativo que realice tareas por nosotros.
- Construir correctamente los comandos, nos permitirá comunicarnos con el sistema operativo y poder utilizar los resultados de estos comandos en nuestras aplicaciones.
- En GNU/Linux, existen programas en modo texto para realizar casi cualquier cosa. En muchos casos, cuando utilizamos una interfaz gráfica, ésta es un frontend del programa en modo comando. Este frontend, puede proporcionar todas o algunas de las funcionalidades de la herramienta real.
- La administración de sistemas, y más si se realiza de forma remota, es más eficiente en modo comando. Las administradoras y administradores de sistemas experimentadas utilizan scripts y modo comandos, tanto en sistemas Windows como GNU/Linux.

El comienzo en el mundo de los comandos, puede resultar aterrador, hay muchísimos comandos, ¡es imposible aprendérselos todos! Bueno, no nos alarmemos, con este par de trucos podremos defendernos:

El nombre de los comandos suele estar relacionado con la tarea que realizan, sólo que expresado en inglés, o utilizando siglas. Por ejemplo: tasklist muestra un listado de los procesos en sistemas Windows; y en GNU/Linux obtendremos el listado de los procesos con ps, que son las siglas de 'process status'.
Su sintaxis siempre tiene la misma forma:

nombreDelComandoopciones
Las opciones, dependen del comando en si. Podemos consultar el manual del comando antes de utilizarlo. En GNU/Linux, lo podemos hacer con "man nombreDelComando"; y en Windows, con "nombreDelComando /?"

Recuerda dejar siempre un espacio en blanco después del nombreDelComando y entre las opciones.

Después de esos pequeños apuntes, los comandos que nos interesa conocer para la gestión de procesos son:

- Windows. Este sistema operativo es conocido por sus interfaces gráficas, el intérprete de comandos conocido como Símbolo del sistema, no ofrece muchos comandos para la gestión de procesos. Tendremos:

- tasklist. Lista los procesos presentes en el sistema. Mostrará el nombre del ejecutable; su correspondiente Identificador de proceso; y, el porcentaje de uso de memoria; entre otros datos
- taskkill. Mata procesos. Con la opción /PID especificaremos el Identificador del proceso que queremos matar.

- GNU/Linux. En este sistema operativo, todo se puede realizar cualquier tarea en modo texto, además de que los desarrolladores y desarrolladoras respetan en la implementación de las aplicaciones, que sus configuraciones se guarden en archivos de texto plano. Esto es muy útil para las administradoras y administradores de sistemas.
- ps. Lista los procesos presentes en el sistema. Con la opción "aux" muestra todos los procesos del sistema independientemente del usuario que los haya lanzado.
- pstree. Muestra un listado de procesos en forma de árbol, mostrando qué procesos han creado otros. Con la opción "AGu" construirá el árbol utilizando líneas guía y mostrará el nombre de usuario propietario del proceso.
- kill. Manda señales a los procesos. La señal -9, matará al proceso. Se utiliza "kill -9 ".
- killall. Mata procesos por su nombre. Se utiliza como "killall nombreDeAplicacion".
- nice. Cambia la prioridad de un proceso. "nice -n 5 comando" ejecutará el comando con una prioridad 5. Por defecto la prioridad es 0. Las prioridades están entre -20 (más alta) y 19 (más baja).

Veremos al final del siguiente apartado un ejemplo, en el que comprobaremos si, realmente, nuestro proyecto CrearProcesos creábamos distintos procesos. También daremos respuesta a la pregunta que planteamos en el apartado de la Gestión de procesos por parte del sistema operativo: ¿quién es el que gestiona los procesos java, el sistema operativo o la máquina virtual java?

> Actualmente, está disponible un nuevo interfaz de comandos para los sistemas Windows denominado Powershell. Incrementa la potencia que tiene el intérprete de comandos común, así se recupera la potencia de los scripts de administración. En los sistemas Windows, ya viene preinstalado. Podemos lanzar su intérprete de comandos (Símbolo de Windows → Todos los programas → Accesorios → Windows Powershell) y aprender su sintaxis (un nuevo lenguaje de programación).

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Herramientas gráficas para la gestión de procesos

> Pero, ¿tenemos que hacerlo todo en modo comandos? ¿qué nos permite hacer el Administrador de tareas de Windows con los procesos? ¿No hay ninguna herramienta gráfica similar en los sistemas GNU/Linux?

Tanto los sistemas Windows como GNU/Linux proporcionan herramientas gráficas para la gestión de procesos. En el caso de Windows, se trata del Administrador de tareas, y en GNU/Linux del Monitor del sistema. Ambos, son bastante parecidos, nos ofrecen, al menos, las siguientes funcionalidades e información:

- Listado de todos los procesos que se encuentran activos en el sistema, mostrando su PID, usuario y ubicación de su fichero ejecutable.
- Posibilidad de finalizar procesos.
- Información sobre el uso de CPU, memoria principal y virtual, red, …
- Posibilidad de cambiar la prioridad de ejecución de los procesos.

> SysInternals, es un conjunto de utilidades avanzadas para SO Windows publicadas como freeware. En particular, recomendamos las herramientas gráficas "Process Explorer" y "Process Monitor". "Process Explorer" nos dará información más completa sobre los procesos activos en el sistema; y "Process Monitor" nos informará de la actividad (de E/S) de los procesos e hilos activos en el sistema: ficheros a los que están accediendo, actividad en red, creación de hilos, etc.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Programación concurrente

Hasta ahora hemos programado aplicaciones secuenciales u orientadas a eventos. Siempre hemos pensado en nuestras aplicaciones como si se ejecutaran de forma aislada en la máquina. De hecho, el SO garantiza que un proceso no accede al espacio de trabajo (zona de memoria) de otro, esto es, unos procesos no pueden acceder a las variables de otros procesos. Sin embargo, los procesos, en ocasiones, necesitan comunicarse entre ellos, o necesitan acceder al mismo recurso (fichero, dispositivo, etc.). En esas situaciones, hay que controlar la forma en la que esos procesos se comunican o acceden a los recursos, para que no haya errores, resultados incorrectos o inesperados.

Podemos ver la concurrencia como una carrera, en la que todos los corredores corren al mismo tiempo buscando un mismo fin, que es ganar la carrera. En el caso de los procesos, competirán por conseguir todos los recursos que necesiten.

La definición de concurrencia, no es algo sencillo. En el diccionario, concurrencia es la coincidencia de varios sucesos al mismo tiempo.

Nosotros podemos decir que dos procesos son concurrentes, cuando la primera instrucción de un proceso se ejecuta después de la primera y antes de la última de otro proceso.

Por otro lado, hemos visto que los procesos activos se ejecutan alternando sus instantes de ejecución en la CPU. Y, aunque nuestro equipo tenga más de un núcleo, los tiempos de ejecución de cada núcleo se repartirán entre los distintos procesos en ejecución. La planificación alternando los instantes de ejecución en la gestión de los procesos, hace que los procesos se ejecuten de forma concurrente. O lo que es lo mismo: multiproceso = concurrencia.

> La programación concurrente proporciona mecanismos de comunicación y sincronización entre procesos que se ejecutan de forma simultanea en un sistema informático. La programación concurrente nos permitirá definir qué instrucciones de nuestros procesos se pueden ejecutar de forma simultánea con las de otros procesos, sin que se produzcan errores; y cuáles deben ser sincronizadas con las de otros procesos para que los resultados de sean correctos.

En el resto de la unidad pondremos especial cuidado en estudiar cómo solucionar los conflictos que pueden surgir cuando dos o más procesos intentan acceder al mismo recurso de forma concurrente.

> **Reflexiona**
>
> La naturaleza y los modelos de interacción entre procesos de un programa concurrente, fueron estudiados y descritos por Dijkstra (1968),Brinch Hansen (1973) y Hoare (1974). Estos trabajos constituyeron los principios en que se basaron los sistemas operativos multiproceso de la década de los 70 y 80.

¿Sabías que los sistemas operativos de Microsoft no fueron multiproceso hasta la aparición de Windows 95 (en 1995)?
MS-DOS era un sistema operativo monousuario y monotarea. Los programadores, no el sistema operativo, implementaban la alternancia en la ejecución de las distintas instrucciones de los procesos que constituían su aplicación para conseguir interactuar con el usuario. Lo conseguían capturando las interrupciones hardware del equipo. Además, MS-DOS, no impedía que unos procesos pudieran acceder al espacio de trabajo de otros procesos, e incluso al espacio de trabajo del propio sistema operativo.
Por otro lado, UNIX, que se puede considerar 'antepasado' de los sistemas GNU/Linux, fue diseñado portable, multitarea, multiusuario y en red desde su origen en 1969.

En este caso, las labores del sistema operativo se hacen más complejas, porque los recursos como la memoria siguen siendo compartidos, y hay procesos que son dependientes de otros y, por lo tanto, no se pueden ejecutar simultáneamente.

Ejemplo

Si imaginamos que tenemos una sola puerta de entrada a un recinto cultural, pero manejamos varias colas con sus prioridades (embarazadas, parejas, etc.), hablaríamos de concurrencia. Si además de tener varias colas, tenemos varias puertas de entrada y dinámicamente se puede redirigir, estaríamos hablando de paralelismo.

Tener varias líneas de ejecución no solo afecta al mejor rendimiento de ejecución de procesos, sino que un mismo proceso podría particionarse en líneas de ejecución para mejorar su rendimiento. Este particionamiento puede ser de dos formas:

- Manualmente, el programador puede codificar los hilos en los que un programa se podría dividir.
- Automáticamente, el compilador o la máquina virtual podrían generar hilos atendiendo a las sentencias que son linealmente independientes. Estas técnicas todavía están en vías de desarrollo.

Hasta ahora hemos pensado en la ejecución de los procesos en un único ordenador. Si pensamos en que se pusiera usar más de un ordenador para ejecutar nuestros procesos, haríamos alusión al concepto de programación distribuida.

Los ámbitos de aplicación del cómputo distribuido difieren de los anteriores y suelen ceñirse a cálculos aritméticos pesados y susceptibles de ser troceados para ejecutarse remotamente y, después, unificar su resultado. El impedimento en estos casos es que recursos como la memoria no son compatibles.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Fundamentos de programación concurrente

Un proceso cooperativo es un proceso que puede verse afectado por otros procesos que se estén ejecutando. Estos procesos pueden compartir únicamente datos, intercambiados mediante algún mecanismo de envío de mensajes o mediante el uso de archivos, o pueden compartir datos y código, como ocurre con los hilos o procesos ligeros.
En cualquier caso, el acceso concurrente a datos compartidos puede generar inconsistencias si no se usa algún tipo de esquema que garantice la coherencia de los mismos. A continuación, se discute esta problemática y se plantean brevemente algunas soluciones

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

¿Para qué concurrencia?

Por supuesto, la ejecución de una aplicación de forma secuencial y aislada en una máquina es lo más eficiente para esa aplicación. Entonces, ¿para qué la concurrencia?

Las principales razones por las que se utiliza una estructura concurrente son:

- **Optimizar la utilización de los recursos**. Podremos simultanear las operaciones de E/S en los procesos. La CPU estará menos tiempo ociosa. Un equipo informático es como una cadena de producción, obtenemos más productividad realizando las tareas concurrentemente.
- **Proporcionar interactividad a los usuarios (y animación gráfica)**. Todos nos hemos desesperado esperando que nuestro equipo finalizara una tarea. Esto se agravaría sino existiera el multiprocesamiento, sólo podríamos ejecutar procesos por lotes.
- **Mejorar la disponibilidad**. Servidor que no realice tareas de forma concurrente, no podrá atender peticiones de clientes simultáneamente.
- **Conseguir un diseño conceptualmente más comprensible y mantenible**. El diseño concurrente de un programa nos llevará a una mayor modularidad y claridad. Se diseña una solución para cada tarea que tenga que realizar la aplicación (no todo mezclado en el mismo algoritmo).
- **Cada proceso se activará cuando sea necesario realizar cada tarea**.
- **Aumentar la protección**. Tener cada tarea aislada en un proceso permitirá depurar la seguridad de cada proceso y, poder finalizarlo en caso de mal funcionamiento sin que suponga la caída del sistema.

Los anteriores pueden parecer los motivos para utilizar concurrencia en sistemas con un solo procesador. Los actuales avances tecnológicos hacen necesario tener en cuenta la concurrencia en el diseño de las aplicaciones para aprovechar su potencial. Los nuevos entornos hardware son:

- **Microprocesadores** con **múltiples núcleos** que comparten la memoria principal del sistema.
- Entornos **multiprocesador** con **memoria compartida**. Todos los procesadores utilizan un mismo espacio de direcciones a memoria, sin tener conciencia de dónde están instalados físicamente los módulos de memoria.
- **Entornos distribuidos**. Conjunto de equipos heterogéneos o no, conectados por red y/o Internet.

Los beneficios que obtendremos al adoptar un modelo de programa concurrente son:

- Estructurar un programa como conjunto de procesos concurrentes que interactúan, aporta gran claridad sobre lo que cada proceso debe hacer y cuando debe hacerlo.
- Puede conducir a una reducción del tiempo de ejecución. Cuando se trata de un entorno monoprocesador, permite solapar los tiempos de E/S o de acceso al disco de unos procesos con los tiempos de ejecución de CPU de otros procesos. Cuando el entorno es multiprocesador, la ejecución de los procesos es realmente simultánea en el tiempo (paralela), y esto reduce el tiempo de ejecución del programa.

- Permite una mayor flexibilidad de planificación. Procesos de alta prioridad pueden ser ejecutados antes de otros procesos menos urgentes.
- La concepción concurrente del software permite un mejor modelado previo del comportamiento del programa, y en consecuencia un análisis más fiable de las diferentes opciones que requiera su diseño.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Condiciones de competencia

Acabamos de ver que tenemos que desechar la idea de que nuestra aplicación se ejecutará de forma aislada. Y que, de una forma u otra, va a interactuar con otros procesos. Distinguimos los siguientes tipos básicos de interacción entre procesos concurrentes:

- Independientes. Sólo interfieren en el uso de la CPU.
- Cooperantes. Un proceso genera la información o proporciona un servicio que otro necesita.
- Competidores. Procesos que necesitan usar los mismos recursos de forma exclusiva.

En el segundo y tercer caso, necesitamos componentes que nos permitan establecer acciones de sincronización y comunicación entre los procesos.

Un proceso entra en condición de competencia con otro, cuando ambos necesitan el mismo recurso, ya sea forma exclusiva o no; por lo que será necesario utilizar mecanismos de sincronización y comunicación entre ellos.

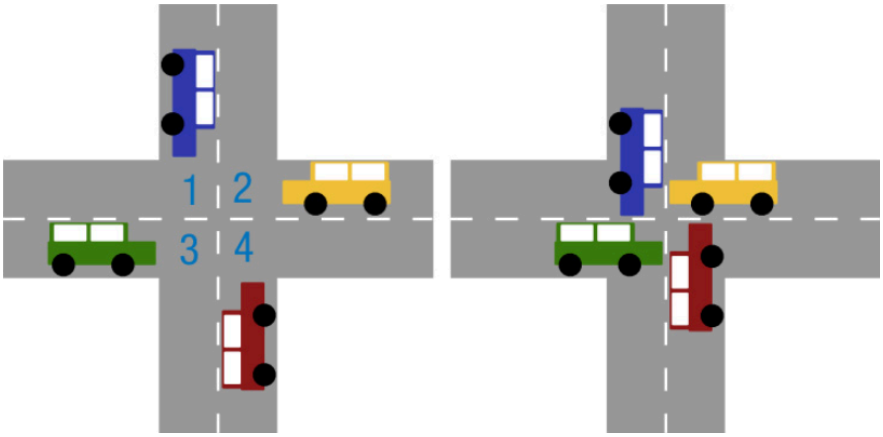
Un ejemplo sencillo de procesos cooperantes, es "un proceso recolector y un proceso productor". El proceso recolector necesita la información que el otro proceso produce. El proceso recolector, quedará bloqueado mientras que no haya información disponible.

El proceso productor, puede escribir siempre que lo desee (es el único que produce ese tipo de información). Por supuesto, podemos complicar esto, con varios procesos recolectores para un sólo productor; y si ese productor puede dar información a todos los recolectores de forma simultánea o no; o a cuántos procesos recolectores puede dar servicio de forma concurrente. Para determinar si los recolectores tendrán que esperar su turno o no. Pero ya abordaremos las soluciones a estas situaciones más adelante.

En el caso de procesos competidores, vamos a comenzar viendo unas definiciones:

- Cuando un proceso necesita un recurso de forma exclusiva, es porque mientras que lo esté utilizando él, ningún otro puede utilizarlo. Se llama región de exclusión mutua o región crítica al conjunto de instrucciones en las que el proceso utiliza un recurso y que se deben ejecutar de forma exclusiva con respecto a otros procesos competidores por ese mismo recurso.
- Cuando más de un proceso necesitan el mismo recurso, antes de utilizarlo tienen que pedir su uso, una vez que lo obtienen, el resto de procesos quedarán bloqueados al pedir ese mismo recurso. Se dice que un proceso hace un lock (bloqueo) sobre un recurso cuando ha obtenido su uso en exclusión mutua.
- Por ejemplo dos procesos, compiten por dos recursos distintos, y ambos necesitan ambos recursos para continuar. Se puede dar la situación en la que cada uno de los procesos bloquee uno de los recursos, lo que hará que el otro proceso no pueda obtener el recurso que le falta; quedando bloqueados un proceso por el otro sin poder finalizar. Deadlock o interbloqueo , se produce cuando los procesos no pueden obtener, nunca, los recursos necesarios para continuar su tarea. El interbloqueo es una situación muy peligrosa, ya que puede llevar al sistema a su caída o cuelgue.

¿Crees que no es usual que pueda darse una situación de interbloqueo? Veamos un ejemplo sencillo: un cruce de caminos y cuatro coches.



El coche azul necesita las regiones 1 y 3 para continuar, el amarillo: 2 y 1, el rojo: 4 y 2, y el verde: 3 y 4.

Obviamente, no siempre quedarán bloqueados, pero se puede dar la situación en la que ninguno ceda. Entonces, quedarán interbloqueados.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

El problema del productor/consumidor

Considere un buffer de tamaño limitado que se comparte por un proceso productor y otro proceso consumidor. Mientras el primero añade elementos al espacio de almacenamiento compartido, el segundo los consume. Evidentemente, el acceso concurrente a este buffer puede generar incosistencias de dichos datos a la hora de, por ejemplo, modificar los elementos contenidos en el mismo.

Suponga que también se mantiene una variable cont que se incrementa cada vez que se añade un elemento al buffer y se decrementa cuando se elimina un elemento del mismo. Este esquema posibilita manejar buffer_size elementos, en lugar de uno menos si dicha problemática se controla con dos variables in y out.

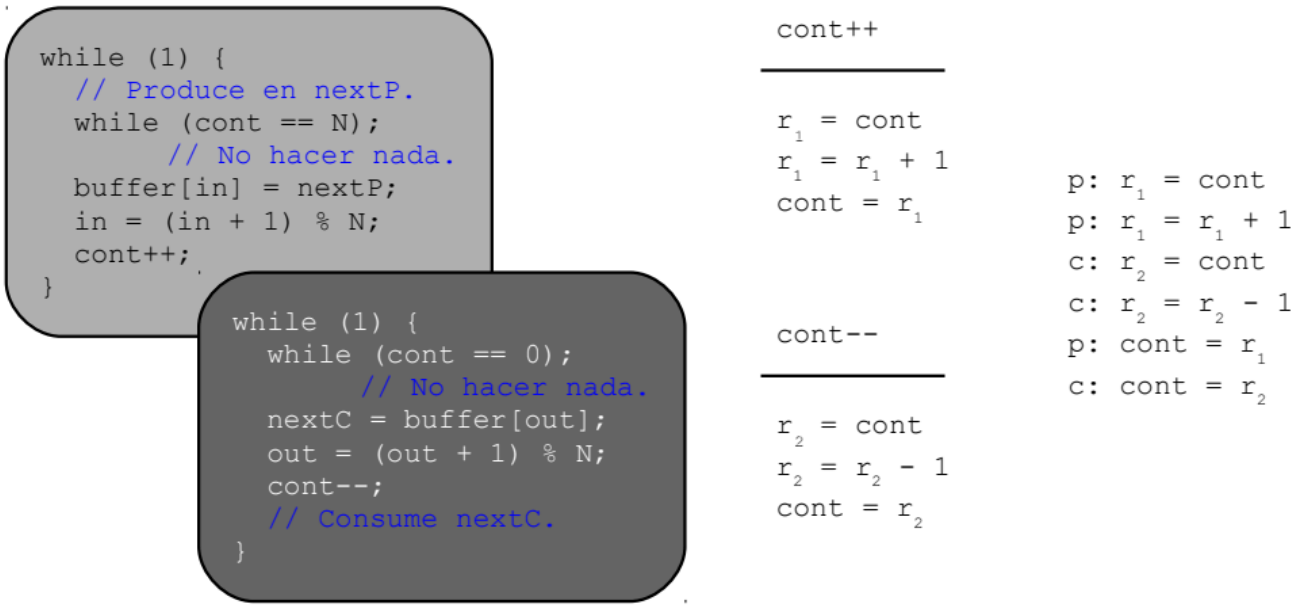
Aunque el código del productor y del consumidor es correcto de manera independiente, puede que no funcione correctamente al ejecutarse de manera simultánea. Por ejemplo, la ejecución concurrente

de las operaciones de incremento y decremento del contador pueden generar inconsistencias, tal y como se aprecia en la figura, en la cual se desglosan dichas operaciones en instrucciones máquina. En dicha figura, el código de bajo nivel asociado a incrementar y decrementar el contador inicializado a 5, genera un resultado inconsistente, ya que esta variable tendría un valor final de 4, en lugar de 5.

Este estado incorrecto se alcanza debido a que la manipulación de la variable contador se realiza de manera concurrente por dos procesos independientes sin que exista ningún mecanismos de sincronización.

Este tipo de situación, en la que se produce un acceso concurrente a unos datos compartidos y el resultado de la ejecución depende del orden de las instrucciones, se denomina condición de carrera. Para evitar este tipo de situaciones se ha de garantizar que sólo un proceso pueda manipular los datos compartidos de manera simultánea, es decir, dichos procesos se han de sincronizar de algún modo.

Este tipo de situaciones se producen constantemente en el ámbito de los sistemas operativos, por lo que el uso de mecanismos de sincronización es crítico para evitar problemas potenciales.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Comunicación entre procesos

Como ya hemos comentado en más de una ocasión a lo largo de esta unidad, cada proceso tiene su espacio de direcciones privado, al que no pueden acceder el resto de procesos. Esto constituye un mecanismo de seguridad; imagina qué locura, si tienes un dato en tu programa y cualquier otro, puede modificarlo de cualquier manera. Tu programa generaría errores, como poco.

Por supuesto, nos damos cuenta de que, si cada proceso tiene sus datos y otros procesos no pueden acceder a ellos directamente, cuando otro proceso los necesite, tendrá que existir alguna forma de comunicación entre ellos.

Comunicación entre procesos: un proceso da o deja información; recibe o recoge información.

Los lenguajes de programación y los sistemas operativos, nos proporcionan primitivas de sincronización que facilitan la interacción entre procesos de forma sencilla y eficiente.

Una primitiva, hace referencia a una operación de la cual conocemos sus restricciones y efectos, pero no su implementación exacta. Veremos que usar esas primitivas se traduce en utilizar objetos y sus métodos, teniendo muy en cuenta sus repercusiones reales en el comportamiento de nuestros procesos.

Clasificaremos las interacciones entre los procesos y el resto del sistema (recursos y otros procesos), como estas tres:

- **Sincronización**: Un proceso puede conocer el punto de ejecución en el que se encuentra otro en ese determinado instante.
- **Exclusión mutua**: Mientras que un proceso accede a un recurso, ningún otro proceso accede al mismo recurso o variable compartida.
- **Sincronización condicional**: Sólo se accede a un recurso cuando se encuentra en un determinado estado interno.

> Por definición, los procesos de un sistema son elementos estancos. Cada uno tiene su espacio de memoria, su tiempo de CPU asignado por el planificador y su estado de los registros. No obstante, los procesos deben poder comunicarse entre sí, ya que es natural que surjan dependencias entre ellos en lo referente a las entradas y salida de datos.
> La comunicación entre procesos se denomina IPC (Inter-Process Communication) y existen diversas alternativas para llevarla a cabo:
>
> - Utilización de **sockets**. Los sockets son mecanismos de comunicación de bajo nivel. Permiten establecer canales de comunicación de bytes bidireccionales entre procesos alojados en distintas máquinas y programados con diferentes lenguajes.
> Gracias a los sockets dos procesos pueden intercambiar cualquier tipo de información.

- Utilización de **flujos de entrada y salida**. Los procesos pueden interceptar los flujos de entrada y salida estándar, por lo que pueden leer y escribir información unos en otros. En este caso, los procesos deben estar relacionados previamente (uno de ellos debe haber arrancado al otro obteniendo una referencia al mismo).
- **RPC**. Llamada a procedimiento remoto (Remote Process Call). Consiste en realizar llamadas a métodos de otros procesos que, potencialmente, pueden estar ejecutándose en otras máquinas. Desde el punto de vista del proceso que realiza la llamada, la ubicación de los procesos llamados es transparente. En Java, este tipo de llamada se realiza mediante tecnología conocida como RMI (Remote Method Invocation), equivalente a las RPC, pero orientadas a objetos.
- Mediante el uso de **sistemas de persistencia**. Consiste en realizar escrituras y lecturas desde los distintos procesos en cualquier tipo de sistemas de persistencia, como los ficheros o las bases de datos. Pese a su sencillez, no se puede ignorar esta alternativa, ya que puede ser suficiente en múltiples ocasiones.
- Mediante el uso de **servicios** proporcionados a través de **internet**. Los procesos pueden utilizar servicios de transferencia de ficheros FTP, aplicaciones o servicios web, así como la tecnología cloud como mecanismos de conexión entre procesos que permitan el intercambio de información.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Mecanismos básicos de comunicación.

Si pensamos en la forma en la que un proceso puede comunicarse con otro. Se nos ocurrirán estas dos:

- Intercambio de mensajes. Tendremos las primitivas enviar (send) y recibir (receive o wait) información.
- Recursos (o memoria) compartidos. Las primitivas serán escribir (write) y leer (read) datos en o de un recurso.

En el caso de comunicar procesos dentro de una misma máquina, el intercambio de mensajes, se puede realizar de dos formas:

- Utilizar un buffer de memoria.
- Utilizar un socket.

La diferencia entre ambos, está en que un socket se utiliza para intercambiar información entre procesos en distintas máquinas a través de la red; y un buffer de memoria, crea un canal de comunicación entre dos procesos utilizando la memoria principal del sistema. Actualmente, es más común el uso de sockets que buffers para comunicar procesos. Trataremos en profundidad los sockets en posteriores unidades. Pero veremos un par de ejemplos muy sencillos de ambos.

En java, utilizaremos sockets y buffers como si utilizáramos cualquier otro stream o flujo de datos. Utilizaremos los métodos read-write en lugar de send-receive.

Con respecto a las lecturas y escrituras, debemos recordar, que serán bloqueantes. Es decir, un proceso quedará bloqueado hasta que los datos estén listos para poder ser leídos. Una escritura, bloqueará al proceso que intenta escribir, hasta que el recurso no esté preparado para poder escribir. Aunque, esto está relacionado con el acceso a recursos compartidos, cosa que estudiaremos en profundidad, en el apartado Regiones críticas.

Pero, esto parece que va a ser algo complicado. ¿Cómo implementamos la comunicación entre procesos?

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Tipos de comunicación

Ya hemos visto que dos procesos pueden comunicarse. Remarquemos algunos conceptos fundamentales sobre comunicación.

En cualquier comunicación, vamos a tener los siguientes elementos:

- Mensaje. Información que es el objeto de la comunicación.
- Emisor. Entidad que emite, genera o es origen del mensaje.
- Receptor. Entidad que recibe, recoge o es destinataria del mensaje.
- Canal. Medio por el que viaja o es enviado y recibido el mensaje.

Podemos clasificar el canal de comunicación según su capacidad, y los sentidos en los que puede viajar la información, como:

- Símplex. La comunicación se produce en un sólo sentido. El emisor es origen del mensaje y el receptor escucha el mensaje al final del canal. Ejemplo: reproducción de una película en una sala de cine.
- Dúplex (Full Duplex). Pueden viajar mensajes en ambos sentidos simultáneamente entre emisor y receptor. El emisor es también receptor y el receptor es también emisor. Ejemplo: telefonía.
- Semidúplex (Half Duplex). El mensaje puede viajar en ambos sentidos, pero no al mismo tiempo. Ejemplo: comunicación con walkie-talkies.

Otra clasificación dependiendo de la sincronía que mantengan el emisor y el receptor durante la comunicación, será:

- Síncrona. El emisor queda bloqueado hasta que el receptor recibe el mensaje. Ambos se sincronizan en el momento de la recepción del mensaje.
- Asíncrona. El emisor continúa con su ejecución inmediatamente después de emitir el mensaje, sin quedar bloqueado.
- Invocación remota. El proceso emisor queda suspendido hasta que recibe la confirmación de que el receptor recibido correctamente el mensaje, después emisor y receptor ejecutarán síncronamente un segmento de código común.

Dependiendo del comportamiento que tengan los interlocutores que intervienen en la comunicación, tendremos comunicación:

- Simétrica. Todos los procesos pueden enviar y recibir información.
- Asimétrica. Sólo un proceso actúa de emisor, el resto sólo escucharán el o los mensajes.

En nuestro anterior ejemplo básico de comunicación con sockets: el proceso SocketEscritor, era el emisor; el proceso SocketLector, era el receptor. El canal de comunicación: sockets. En el ejemplo, hemos utilizado del socket en una sola dirección y síncrona; pero los sockets permiten comunicación dúplex síncrona (en cada sentido de la comunicación) y simétrica (ambos procesos

pueden escribir en y leer del socket); también existen otros tipos de sockets que nos permitirán establecer comunicaciones asimétricas asíncronas (DatagramSocket).

En el caso del ejemplo de las tuberías, la comunicación que se establece es simplex síncrona y asimétrica.

Nos damos cuenta, que conocer las características de la comunicación que necesitamos establecer entre procesos, nos permitirá seleccionar el canal, herramientas y comportamiento más convenientes.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Sincronización entre procesos

Ya tenemos mucho más claro, que las situaciones en las que dos o más procesos tengan que comunicarse, cooperar o utilizar un mismo recurso; implicará que deba haber cierto sincronismo entre ellos. O bien, unos tienen que esperar que otros finalicen alguna acción; o, tienen que realizar alguna tarea al mismo tiempo.

En este capítulo, veremos distintas problemáticas, primitivas y soluciones de sincronización necesarias para resolverlas. También es cierto, que el sincronismo entre procesos lo hace posible el SO, y lo que hacen los lenguajes de programación de alto nivel es encapsular los mecanismos de sincronismo que proporciona cada SO en objetos, métodos y funciones. Los lenguajes de programación, proporcionan primitivas de sincronismo entre los distintos hilos que tenga un proceso; estas primitivas del lenguaje, las veremos en la siguiente unidad.

Comencemos viendo un ejemplo muy sencillo de un problema que se nos plantea de forma más o menos común: inconsistencias en la actualización de un valor compartido por varios procesos; así, nos daremos cuenta de la importancia del uso de mecanismos de sincronización.

En programación concurrente, siempre que accedamos a algún recurso compartido (eso incluye a los ficheros), debemos tener en cuenta las condiciones en las que nuestro proceso debe hacer uso de ese recurso: ¿será de forma exclusiva o no? Lo que ya definimos anteriormente como condiciones de competencia.

En el caso de lecturas y escrituras en un fichero, debemos determinar si queremos acceder al fichero como sólo lectura; escritura; o lectura-escritura; y utilizar los objetos que nos permitan establecer los mecanismos de sincronización necesarios para que un proceso pueda bloquear el uso del fichero por otros procesos cuando él lo esté utilizando.

Esto se conoce como el problema de los procesos lectores-escritores. El sistema operativo, nos ayudará a resolver los problemas que se plantean; ya que:

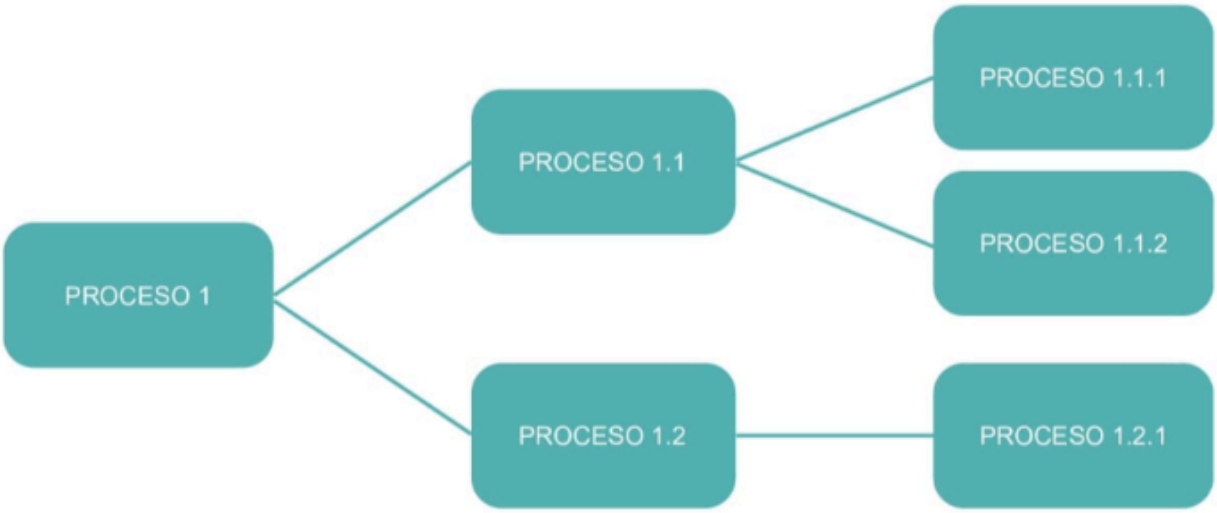
- Si el acceso es de sólo lectura. Permitirá que todos los procesos lectores, que sólo quieren leer información del fichero, puedan acceder simultáneamente a él.
- En el caso de escritura, o lectura-escritura. El SO nos permitirá pedir un tipo de acceso de forma exclusiva al fichero. Esto significará que el proceso deberá esperar a que otros procesos lectores terminen sus accesos. Y otros procesos (lectores o escritores), esperarán a que ese proceso escritor haya finalizado su escritura.

Debemos tener en cuenta que, nosotros, nos comunicamos con el SO a través de los objetos y métodos proporcionados por un lenguaje de programación; y, por lo tanto, tendremos que consultar cuidadosamente la documentación de las clases que estamos utilizando para conocer todas las peculiaridades de su comportamiento.

En la siguiente presentación, podemos ver cómo implementamos dos aplicaciones. Una de ellas, lee un valor de un fichero y lo escribe en el mismo fichero después de incrementarlo en uno. Otra aplicación crea un grupo de procesos de la primera aplicación; todos esos procesos accederán al mismo fichero para realizar la misma acción. Al final de la ejecución, al abrir el fichero que han estado utilizando, el valor que encontremos, ¿será el que esperamos que debe ser?



Todos los sistemas en los que participan múltiples actores de manera concurrente están sometidos a ciertas condiciones que exigen que exista sincronización entre ellos. Por ejemplo, puede que sea necesario saber si un proceso ha terminado satisfactoriamente para ejecutar el siguiente que se encuentra en un flujo de procesos o, en caso de que haya ocurrido un determinado error, ejecutar otro proceso alternativo.
Es el planificador del sistema operativo el encargado de decidir en qué formato tiene acceso a los recursos un proceso, pero a nivel general, la decisión de crear y lanzar un proceso es humana y expresada a través de un algoritmo.
En la siguiente imagen se muestra un posible ejemplo de flujo de ejecución de un conjunto de procesos. Las condiciones que determinan dicho flujo son las siguientes:
El proceso "Proceso 1" se ejecuta inicialmente.
Si el código de finalización de "Proceso 1" es 0, se ejecuta el proceso "Proceso 1.1".
Si el código de finalización de "Proceso 1.1" es 0, se ejecuta el "Proceso 1.1.1".
Si el código de finalización de "Proceso 1.1" es 1, se ejecuta el proceso "Proceso 1.1.2".
Si el código de finalización de "Proceso 1" es 1, se ejecuta el proceso "Proceso 1.2".
Independientemente del código de finalización del proceso "Proceso 1.2", pero únicamente cuando haya finalizado, se ejecuta el proceso "Proceso 1.2.1"



Todos los sistemas en los que participan múltiples actores de manera concurrente están sometidos a ciertas condiciones que exigen que exista sincronización entre ellos. Por ejemplo, puede que sea necesario saber si un proceso ha terminado satisfactoriamente para ejecutar el siguiente que se encuentra en un flujo de procesos o, en caso de que haya ocurrido un determinado error, ejecutar otro proceso alternativo.
Es el planificador del sistema operativo el encargado de decidir en qué formato tiene acceso a los recursos un proceso, pero a nivel general, la decisión de crear y lanzar un proceso es humana y expresada a través de un algoritmo.
En la siguiente imagen se muestra un posible ejemplo de flujo de ejecución de un conjunto de procesos. Las condiciones que determinan dicho flujo son las siguientes:
El proceso "Proceso 1" se ejecuta inicialmente.
Si el código de finalización de "Proceso 1" es 0, se ejecuta el proceso "Proceso 1.1".
Si el código de finalización de "Proceso 1.1" es 0, se ejecuta el "Proceso 1.1.1".
Si el código de finalización de "Proceso 1.1" es 1, se ejecuta el proceso "Proceso 1.1.2".
Si el código de finalización de "Proceso 1" es 1, se ejecuta el proceso "Proceso 1.2".
Independientemente del código de finalización del proceso "Proceso 1.2", pero únicamente cuando haya finalizado, se ejecuta el proceso "Proceso 1.2.1"

Este tipo de sincronización se puede realizar entre procesos escritos en distintos lenguajes de programación

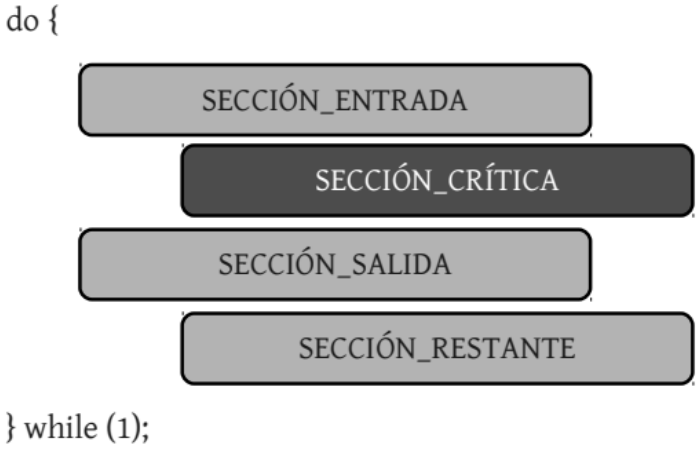
Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Regiones críticas

El segmento de código en el que un proceso puede modificar variables compartidas con otros procesos se denomina sección o región crítica. Para evitar inconsistencias, una de las ideas que se plantean es que cuando un proceso está ejecutando su región crítica ningún otro procesos puede ejecutar su región crítica asociada.

El problema de la sección crítica consiste en diseñar algún tipo de solución para garantizar que los procesos involucrados puedan operar sin generar ningún tipo de inconsistencia. Una posible estructura para abordar esta problemática se plantea en la siguiente figura, en la que el código se divide en las siguientes secciones:

- Sección de entrada, en la que se solicita el acceso a la sección crítica.
- Sección crítica, en la que se realiza la modificación efectiva de los datos compartidos.
- Sección de salida, en la que típicamente se hará explícita la salida de la sección crítica.
- Sección restante, que comprende el resto del código fuente.



Normalmente, la sección de entrada servirá para manipular algún tipo de mecanismo de sincronización que garantice el acceso exclusivo a la sección crítica de un proceso. Mientras tanto, la sección de salida servirá para notificar, mediante el mecanismo de sincronización correspondiente, la salida de la sección crítica. Este hecho, a su vez, permitirá que otro proceso pueda acceder a su sección crítica.
Cualquier solución al problema de la sección crítica ha de satisfacer los siguientes requisitos:

1. Exclusión mutua, de manera que si un proceso pi está en su sección crítica, entonces ningún otro proceso puede ejecutar su sección crítica.
2. Progreso, de manera que sólo los procesos que no estén en su sección de salida, suponiendo que ningún proceso ejecutase su sección crítica, podrán participar en la decisión de quién es el siguiente en ejecutar su sección crítica. Además, la toma de esta decisión ha de realizarse en un tiempo limitado.
3. Espera limitada, de manera que existe un límite en el número de veces que se permite entrar a otros procesos a sus secciones críticas antes de que otro proceso haya solicitado entrar en su propia sección crítica (y antes de que le haya sido concedida).

Las soluciones propuestas deberían ser independientes del número de procesos, del orden en el que se ejecutan las instrucciones máquinas y de la velocidad relativa de ejecución de los procesos.

La definición común, y que habíamos visto anteriormente, de una región o sección crítica, es, el conjunto de instrucciones en las que un proceso accede a un recurso compartido. Para que la definición sea correcta, añadiremos que, las instrucciones que forman esa región crítica, se ejecutarán de forma indivisible o atómica y de forma exclusiva con respecto a otros procesos que accedan al mismo recurso compartido al que se está accediendo.

Al identificar y definir nuestras regiones críticas en el código, tendremos en cuenta:

- Se protegerán con secciones críticas sólo aquellas instrucciones que acceden a un recurso compartido.
- Las instrucciones que forman una sección crítica, serán las mínimas. Incluirán sólo las instrucciones imprescindibles que deban ser ejecutadas de forma atómica.
- Se pueden definir tantas secciones críticas como sean necesarias.
- Un único proceso entra en su sección crítica. El resto de procesos esperan a que éste salga de su sección crítica. El resto de procesos esperan, porque encontrarán el recurso bloqueado. El proceso que está en su sección crítica, es el que ha bloqueado el recurso.
- Al final de cada sección crítica, el recurso debe ser liberado para que puedan utilizarlo otros procesos.

Algunos lenguajes de programación permiten definir bloques de código como secciones críticas. Estos lenguajes, cuentan con palabras reservadas específicas para la definición de estas regiones. En Java, veremos cómo definir este tipo de regiones a nivel de hilo en posteriores unidades.

A nivel de procesos, lo primero, haremos, que nuestro ejemplo de accesos múltiples a un fichero, sea correcto para su ejecución en un entorno concurrente. En esta presentación identificaremos la sección o secciones críticas y qué objetos debemos utilizar para conseguir que esas secciones se ejecuten de forma excluyente.



En nuestro ejemplo, hemos visto cómo definir una sección crítica para proteger las actualizaciones de un fichero. Cualquier actualización de datos en un recurso compartido, necesitará establecer una región crítica que implicará como mínimo estas instrucciones:

- Leer el dato que se quiere actualizar. Pasar el dato a la zona de memoria local al proceso.

- Realizar el cálculo de actualización. Modificar el dato en memoria.
- Escribir el dato actualizado. Llevar el dato modificado de memoria al recurso compartido.

Debemos darnos cuenta de que nos referimos a un recurso compartido de forma genérica, ese recurso compartido podrá ser: memoria principal, fichero, base de datos, etc.

> Además de bloquear un fichero completo, podemos solicitar al sistema que bloquee sólo una región del fichero, por lo que varios procesos pueden actualizar al mismo tiempo el fichero mientras que estén actualizando información que se encuentre en zonas distintas y no solapadas del fichero.
>
> 
>
> *Ampliar información sobre los métodos lock() de la clase FileChannel.*

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Categoría de proceso cliente-suministrador

En este caso, vamos a hacer una introducción a los procesos que podremos clasificar dentro de la categoría cliente-suministrador.

- Cliente. Es un proceso que requiere o solicita información o servicios que proporciona otro proceso.
- Suministrador. Probablemente, te suene más el término servidor; pero, no queremos confundirnos con el concepto de servidor en el que profundizaremos en próximas unidades. Suministrador, hace referencia a un concepto de proceso más amplio; un suministrador, suministra información o servicios; ya sea a través memoria compartida, un fichero, red, o cualquier otro recurso.
- Información o servicio es perecedero. La información desaparece cuando es consumida por el cliente; y, el servicio es prestado en el momento en el que cliente y suministrador están sincronizados.

Entre un cliente y un suministrador (ojo, empecemos con un proceso de cada), se establece sincronismo entre ellos, por medio de intercambio de mensajes o a través de un recurso compartido. Entre un cliente y un servidor, la comunicación se establece de acuerdo a un conjunto mensajes a intercambiar con sus correspondientes reglas de uso; llamado protocolo. Podremos implementar nuestros propios protocolos, o, protocolos existentes (ftp, http, telnet, smtp, pop3, …); pero aún tenemos que ver algunos conceptos más antes de implementar protocolos.

Cliente y suministrador, son, los procesos que vimos en nuestros ejemplos de uso básico de sockets y comunicación a través de tuberías (apartado 5.1. Mecanismos básicos de comunicación); y, por supuesto, se puede extender a los casos en los que tengamos un proceso que lee y otro que escribe en un recurso compartido.

Entre procesos cliente y suministrador debemos disponer de mecanismos de sincronización que permitan que:

- Un cliente no debe poder leer un dato hasta que no haya sido completamente suministrado. Así nos aseguraremos de que el dato leído es correcto y consistente.
- Un suministrador irá produciendo su información, que en cada instante, no podrá superar un volumen de tamaño máximo establecido; por lo que el suministrador, no debe poder escribir un dato si se ha alcanzado ese máximo. Esto es así, para no desbordar al cliente.

Lo más sencillo, es pensar que el suministrador sólo produce un dato que el cliente tiene que consumir. ¿Qué sincronismo hace falta en esta situación?

1. El cliente tiene que esperar a que el suministrador haya generado el dato.
2. El suministrador genera el dato y de alguna forma avisa al cliente de que puede consumirlo.

Podemos pensar en dar una solución a esta situación con programación secuencial. Incluyendo un bucle en el cliente en el que esté testeando el valor de una variable que indica que el dato ha sido

producido.

Como podemos ver en este gráfico el pseudocódigo del cliente incluye el bucle del que habíamos mencionado. Ese bucle hace que esta solución sea poco eficiente, ya que el proceso cliente estaría consumiendo tiempo de CPU sin realizar una tarea productiva; lo que conocemos como espera activa. Además, si el proceso suministrador quedara bloqueado por alguna razón, ello también bloquearía al proceso cliente.

En los próximos apartados, vamos a centrarnos en los mecanismos de programación concurrente que nos permiten resolver estos problemas de sincronización entre procesos de forma eficiente, llamados primitivas de programación concurrente: semáforos y monitores; y son estas primitivas las que utilizaremos para proteger las secciones críticas de nuestros procesos.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Semáforos

Veamos una primera solución eficiente a los problemas de sincronismo, entre procesos que acceden a un mismo recurso compartido.

Podemos ver varios procesos que quieren acceder al mismo recurso, como coches que necesitan pasar por un cruce de calles. En nuestros cruces, los semáforos nos indican cuándo podemos pasar y cuándo no. Nosotros, antes de intentar entrar en el cruce, primero miramos el color en el que se encuentra el semáforo, y si está en verde (abierto), pasamos. Si el color es rojo (cerrado), quedamos a la espera de que ese mismo semáforo nos indique que podemos pasar. Este mismo funcionamiento es el que van a seguir nuestros semáforos en programación concurrente. Y, son una solución eficiente, porque los procesos quedarán bloqueados (y no en espera activa) cuando no puedan acceder al recurso, y será el semáforo el que vaya desbloqueándolos cuando puedan pasar.

> **Un semáforo, es un componente de bajo nivel de abstracción que permite arbitrar los accesos a un recurso compartido en un entorno de programación concurrente.**

Al utilizar un semáforo, lo veremos como un tipo dato, que podremos instanciar. Ese objeto semáforo podrá tomar un determinado conjunto de valores y se podrá realizar con él un conjunto determinado de operaciones. Un semáforo, tendrá también asociada una lista de procesos suspendidos que se encuentran a la espera de entrar en el mismo.

Dependiendo del conjunto de datos que pueda tomar un semáforo, tendremos:

- **Semáforos binarios**. Aquellos que pueden tomar sólo valores 0 ó 1. Como nuestras luces verde y roja.
- **Semáforos generales**. Pueden tomar cualquier valor Natural (entero no negativo).

En cualquier caso, los valores que toma un semáforo representan:

- Valor igual a 0. Indica que el semáforo está cerrado.
- Valor mayor de 0. El semáforo está abierto.

Un semáforo, en términos de programación, es un mecanismo de sincronización que encapsula una variable entera que sólo es accesible mediante dos **operaciones atómicas** estándar: **wait** y **signal.** Estas dos operaciones son **operaciones seguras** (la implementación del semáforo garantiza que la operación de chequeo del valor del semáforo, y posterior actualización según proceda, es siempre segura respecto a otros accesos concurrentes):

- **objSemaforo.wait()**: Si el semáforo no es nulo (está abierto) decrementa en uno el valor del semáforo. Si el valor del semáforo es nulo (está cerrado), el proceso que lo ejecuta se suspende y se encola en la lista de procesos en espera del semáforo.
- **objSemaforo.signal()**: Si hay algún proceso en la lista de procesos del semáforo, activa uno de ellos para que ejecute la sentencia que sigue al wait que lo suspendió. Si no hay procesos en espera en la lista incrementa en 1 el valor del semáforo.

La modificación del valor entero encapsulado dentro de un semáforo ha de modificarse mediante **instrucciones atómicas**, es decir, tanto wait como signal han de ser instrucciones atómicas. Típicamente, estas operaciones se utilizarán para gestionar el **acceso a la sección crítica** por un número indeterminado de procesos.

Además de la operación segura anterior, con un semáforo, también podremos realizar una operación no segura, que es la inicialización del valor del semáforo. Ese valor indicará cuántos procesos pueden entrar concurrentemente en él. Esta inicialización la realizaremos al crear el semáforo.

En Java la implementación de la función wait es la siguiente:

Código Java

```
// Implementación de la operación 'wait' (equivalente a acquire)

public synchronized void waitSemaforo() throws InterruptedException {

while (valor <= 0) {

// Si el valor del semáforo es 0 o negativo, espera

wait();

}

valor--; // Decrementamos el valor del semáforo (adquiere recurso)

}
```

Por otra parte, la definición de signal es la siguiente:

Código Java

```
// Implementación de la operación 'signal' (equivalente a release)

public synchronized void signalSemaforo() {

valor++; // Incrementamos el valor del semáforo (libera recurso)
```

```
notify(); // Despertamos a un hilo que esté esperando en 'waitSemaforo'

}
```

La clase que puede representar el funcionamiento de un Semáforo en Java, quedaría así:

Código Java

```
public class SemaforoPersonalizado {

private int valor;

// Constructor para inicializar el semáforo con un valor inicial

public SemaforoPersonalizado(int valorInicial) {

this.valor = valorInicial;

}

// Implementación de la operación 'wait' (equivalente a acquire)

public synchronized void waitSemaforo() throws InterruptedException {

while (valor <= 0) {

// Si el valor del semáforo es 0 o negativo, espera

wait();

}

valor--; // Decrementamos el valor del semáforo (adquiere recurso)

}

// Implementación de la operación 'signal' (equivalente a release)

public synchronized void signalSemaforo() {

valor++; // Incrementamos el valor del semáforo (libera recurso)

notify(); // Despertamos a un hilo que esté esperando en 'waitSemaforo'

}

}
```

Para utilizar semáforos, seguiremos los siguientes pasos:

1. Un proceso padre creará e inicializará tanto semáforo.
2. El proceso padre creará el resto de procesos hijo pasándoles el semáforo que ha creado. Esos procesos hijos acceden al mismo recurso compartido.
3. Cada proceso hijo, hará uso de las operaciones seguras wait y signal respetando este esquema:
    1. objSemaforo.wait(); Para consultar si puede acceder a la sección crítica.
    2. Sección crítica; Instrucciones que acceden al recurso protegido por el semáforo objSemaforo.
    3. objSemaforo.signal(); Indicar que abandona su sección y otro proceso podrá entrar.
    4. El proceso padre habrá creado tantos semáforos como tipos secciones críticas distintas se puedan distinguir en el funcionamiento de los procesos hijos (puede ocurrir, uno por cada recurso compartido).

A continuación se enumeran algunas características de los semáforos y las implicaciones que tiene su uso en el contexto de la sincronización entre procesos.

- Sólo es posible acceder a la variable del semáforo mediante wait o signal. No se debe asignar o comparar los valores de la variable encapsulada en el mismo.
- Los semáforos han de inicializarse con un valor no negativo.
- La operación wait decrementa el valor del semáforo. Si éste se hace negativo, entonces el proceso que ejecuta wait se bloquea.
- La operación signal incrementa el valor del semáforo. Si el valor no era positivo, entonces se desbloquea a un proceso bloqueado previamente por wait.

Estas características tienen algunas implicaciones importantes. Por ejemplo, no es posible determinar a priori si un proceso que ejecuta wait se quedará bloqueado o no tras su ejecución. Así mismo, no es
posible conocer si después de signal se despertará algún proceso o no.
Por otra parte, wait y signal son mutuamente excluyentes, es decir, si se ejecutan de manera concurrente, entonces se ejecutarán secuencialmente y en un orden no conocido a priori

La ventaja de utilizar semáforos es que son fáciles de comprender, proporcionan una gran capacidad funcional (podemos utilizarlos para resolver cualquier problema de concurrencia). Pero, su nivel bajo de abstracción, los hace peligrosos de manejar y, a menudo, son la causa de muchos errores, como es el interbloqueo. Un simple olvido o cambio de orden conduce a bloqueos; y requieren que la gestión de un semáforo se distribuya por todo el código lo que hace la depuración de los errores en su gestión es muy difícil.

En java, encontramos la clase Semaphore dentro del paquete java.util.concurrent; y su uso real se aplica a los hilos de un mismo proceso, para arbitrar el acceso de esos hilos de forma concurrente a una misma región de la memoria del proceso. Por ello, veremos ejemplos de su uso en siguientes unidades de este módulo.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Monitores

Los monitores, nos ayudan a resolver las desventajas que encontramos en el uso de semáforos. El problema en el uso de semáforos es que, recae sobre el programador o programadora la tarea implementar el correcto uso de cada semáforo para la protección de cada recurso compartido; y, sigue estando disponible el recurso para utilizarlo sin la protección de un semáforo. Los monitores son como guardaespaldas, encargados de la protección de uno o varios recursos específicos; pero encierran esos recursos de forma que el proceso sólo puede acceder a esos recursos a través de los métodos que el monitor expone.

> **Un monitor, es un componente de alto nivel de abstracción destinado a gestionar recursos que van a ser accedidos de forma concurrente.**

Los monitores encierran en su interior los recursos o variables compartidas como componentes privadas y garantizan el acceso a ellas en exclusión mutua.

La declaración de un monitor incluye:

- Declaración de las constantes, variables, procedimientos y funciones que son privados del monitor (solo el monitor tiene visibilidad sobre ellos).
- Declaración de los procedimientos y funciones que el monitor expone (públicos) y que constituyen la interfaz a través de las que los procesos acceden al monitor.
- Cuerpo del monitor, constituido por un bloque de código que se ejecuta al ser instanciado o inicializado el monitor. Su finalidad es inicializar las variables y estructuras internas del monitor.
- El monitor garantiza el acceso al código interno en régimen de exclusión mutua.
- Tiene asociada una lista en la que se incluyen los procesos que al tratar de acceder al monitor son suspendidos.

Los paquetes Java, no proporcionan una implementación de clase Monitor (habría que implementar un monitor para cada variable o recurso a sincronizar). Pero, siempre podemos implementarnos nuestra propia clase monitor, haciendo uso de semáforos para ello.

Pensemos un poco, ¿hemos utilizado objetos que pueden encajar con la declaración de un monitor aunque su tipo de dato no fuera monitor?, ¿no?, ¿seguro? Cuando realizamos una lectura o escritura en fichero, nuestro proceso queda bloqueado hasta que el sistema ha realizado completamente la operación. Nosotros inicializamos el uso del fichero indicando su ruta al crear el objeto, por ejemplo, FileReader; y utilizamos los métodos expuestos por ese objeto para realizar las operaciones que deseamos con ese fichero. Sin embargo, el código que realmente realiza esas operaciones es el implementado en la clase FileReader. Si bien, esos objetos no proporcionan exclusión mutua en los accesos al recurso; o, por lo menos, no en todos sus métodos. Aún así,

podemos decir que utilicemos objetos de tipo monitor al acceder a los recursos del sistema, aunque no tengan como nombre Monitor.

Las ventajas que proporciona el uso de monitores son:

- Uniformidad: El monitor provee una única capacidad, la exclusión mutua, no existe la confusión de los semáforos.
- Modularidad: El código que se ejecuta en exclusión mutua está separado, no mezclado con el resto del programa.
- Simplicidad: El programador o programadora no necesita preocuparse de las herramientas para la exclusión mutua .
- Eficiencia de la implementación: La implementación subyacente puede limitarse fácilmente a los semáforos.

Y, la desventaja:

- Interacción de múltiples condiciones de sincronización: Cuando el número de condiciones crece, y se hacen complicadas, la complejidad del código crece de manera extraordinaria.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Monitores: Lecturas y escrituras bloqueantes en recursos compartidos

Recordemos, el funcionamiento de los procesos cliente y suministrador podría ser el siguiente:

- Utilizan un recurso del sistema a modo de buffer compartido en el que, el suministrador introduce elementos; y, el cliente los extrae.
- Se sincronizarán utilizando una variable compartida que indica el número de elementos que contiene ese buffer compartido, cuyo tamaño máximo será N.
- El proceso suministrador, siempre comprueba antes de introducir un elemento, que esa variable tenga un valor menor que N. Al introducir un elemento incrementa en uno la variable compartida.
- El proceso cliente, extraerá un elemento del buffer y decrementará el valor de la variable; siempre que el valor de la variable indique que hay elementos que consumir.

Los mecanismos de sincronismo que nos permiten el anterior funcionamiento entre procesos, son las lecturas y escrituras bloqueantes en recursos compartidos del sistema (streams). En el caso de java, disponemos de:

- Arquitectura java.io.
  - Implementación de clientes: Para sus clases derivadas de Reader como son InputStream, InputStreamReader, FileReader, …; los métodos read(buffer) y read(buffer, desplazamiento, tamaño).
  - Implementación de suministradores: Con sus análogos derivados de Writer; los métodos write(info) y write(info, desplazamiento, tamaño).
- Arquitectura java.nio (disponible desde la versión 1.4 de Java). Dentro de java.nio.channels:
  - Implementación de clientes: Sus clases FileChannel y SocketChannel; los métodos read(buffer) y read(buffer, desplazamiento, tamaño).
  - Implementación de suministradores: Sus clases FileChannel y SocketChannel; los métodos write(info) y write(info, desplazamiento, tamaño).

> **Advertencia**
>
> Y recordemos que, como vimos en el apartado Regiones críticas; tendremos que hacer uso del método lock() de FileChannel; para implementar las secciones críticas de forma correcta. Tanto para suministradores como para clientes, cuando estemos utilizando un fichero como canal de comunicación entre ellos

> **Código**



Código



Si nos damos cuenta, hasta ahora sólo hemos hablado de un proceso Suministrador y un proceso cliente. El caso en el que un suministrador tenga que dar servicio a más de un cliente, aprenderemos a solucionarlo utilizando hilos o threads, e implementando esquemas de cliente-servidor, en las próximas unidades. No obstante, debemos tener claro, que el sincronismo cuando hay una información que genera un proceso (o hilo), y que recolecta otro proceso (o hilo) atenderá a las características que hemos descrito en estos aparatados.

Saber más

Dependiendo de la aplicación que vayamos a implementar, podemos encontrarnos en la necesidad de que el cliente pueda realizar otras operaciones mientras que no haya datos disponibles. Es decir, que las operaciones de lectura no sean bloqueantes. Un ejemplo claro, es la programación de un cliente de streaming de música o vídeo. Ese cliente es un reproductor on-line que comienza a reproducir los datos que recibe, sin esperar a que se haya descargado completamente el archivo. Por supuesto, en estas situaciones tendremos que evaluar las variantes (con respecto a las que hemos visto en esta unidad) en el sincronismo de los procesos que se comunican.
Para conseguir lecturas no bloqueantes, haremos uso de la arquitectura java.nio, que implementa canales sobre sockets (SocketChannel). Podremos configurar que las lecturas en un SocketChannel no sean bloqueantes por medio del método configureBlocking(booleano), cuando booleano sea false.
Las lecturas en un canal creado sobre un fichero (FileSocket) siempre son bloqueantes.



Ampliar información sobre el método configureBlocking(bool) de la clase SocketChannel.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Memoria compartida

Una forma natural de comunicación entre procesos es la posibilidad de disponer de zonas de memoria compartidas (variables, buffers o estructuras). Además, los mecanismos de sincronización en programación concurrente que hemos visto: regiones críticas, semáforos y monitores; tienen su razón de ser en la existencia de recursos compartidos; incluida la memoria compartida.

Cuando se crea un proceso, el sistema operativo le asigna los recursos iniciales que necesita, siendo el principal recurso: la zona de memoria en la que se guardarán sus instrucciones, datos y pila de ejecución. Pero como ya hemos comentado anteriormente, los sistemas operativos modernos, implementan mecanismos que permiten proteger la zona de memoria de cada proceso siendo ésta privada para cada proceso, de forma que otros no podrán acceder a ella. Con esto, podemos pensar que no hay posibilidad de tener comunicación entre procesos por medio de memoria compartida. Pues, no es así. En la actualidad, la programación multihilo (que abordaremos en la siguiente unidad y, se refiere, a tener varios flujos de ejecución dentro de un mismo proceso, compartiendo entre ellos la memoria asignada al proceso), nos permitirá examinar al máximo esta funcionalidad.

Pensemos ahora en problemas que pueden resultar complicados si los resolvemos con un sólo procesador, por ejemplo: la ordenación de los elementos de una matriz. Ordenar una matriz pequeña, no supone mucho problema; pero si la matriz se hace muy muy grande... Si disponemos de varios procesadores y somos capaces de partir la matriz en trozos (convertir un problema grande en varios más pequeños) de forma que cada procesador se encargue de ordenar cada parte de la matriz. Conseguiremos resolver el problema en menos tiempo; eso sí, teniendo en cuenta la complejidad de dividir el problema y asignar a cada procesador el conjunto de datos (o zona de memoria) que tiene que manejar y la tarea o proceso a realizar (y finalizar con la tarea de combinar todos los resultados para obtener la solución final). En este caso, tenemos sistemas multiprocesador como los actuales microprocesadores de varios núcleos, o los supercomputadores formados por múltiples ordenadores completos (e idénticos) trabajando como un único sistema. En ambos casos, contaremos con ayuda de sistemas específicos (sistemas operativos o entornos de programación), preparados para soportar la carga de computación en múltiples núcleos y/o equipos.

Saber más

Aunque en próximos apartados haremos una introducción a la programación paralela, es interesante conocer la existencia de OpenMP en este apartado. Es, una API para la programación multiproceso de memoria compartida en múltiples plataformas. Es un estándar disponible en muchas arquitecturas, incluidas las plataformas de Unix y de Microsoft Windows. Se compone de un conjunto de directivas de compilador, rutinas de biblioteca, y variables de entorno que influencian el comportamiento en tiempo de

ejecución. OpenMP es un modelo de programación portable y escalable que proporciona a los programadores una interfaz simple y flexible para el desarrollo de aplicaciones paralelas para las plataformas que van desde las computadoras de escritorio hasta las supercomputadoras. También existe una implementación de OpenMP en Java.



Ampliar información sobre OpenMP.

Reflexiona

Hay problemas complejos cuya solución se está haciendo más asequible gracias a la evolución de la tecnología y la programación concurrente. Piensa, en las distintas variantes que hay que evaluar en una predicción meteorológica: temperatura, humedad, vectores de desplazamiento de las masas de aire (y a nivel mundial),... Gracias a los supercomputadores, las predicciones meteorológica son cada día más fiables.



Modelo numérico de predicción meteorológica.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Cola de mensajes

El paso de mensajes es una técnica empleada en programación concurrente para aportar sincronización entre procesos y permitir la exclusión mutua, de manera similar a como se hace con los semáforos, monitores, etc. Su principal característica es que no precisa de memoria compartida.

Los elementos principales que intervienen en el paso de mensajes son el proceso que envía, el que recibe y el mensaje.

Dependiendo de si el proceso que envía el mensaje espera a que el mensaje sea recibido, se puede hablar de paso de mensajes síncrono o asíncrono:

- En el paso de mensajes asíncrono, el proceso que envía, no espera a que el mensaje sea recibido, y continúa su ejecución, siendo posible que vuelva a generar un nuevo mensaje y a enviarlo antes de que se haya recibido el anterior. Por este motivo se suelen emplear buzones o colas, en los que se almacenan los mensajes a espera de que un proceso los reciba. Generalmente empleando este sistema, el proceso que envía mensajes sólo se bloquea o para, cuando finaliza su ejecución, o si el buzón está lleno. Para conseguir esto, estableceremos una serie de reglas de comunicación (o protocolo) entre emisor y receptor, de forma que el receptor pueda indicar al emisor qué capacidad restante queda en su cola de mensajes y si está lleno o no.
- En el paso de mensajes síncrono, el proceso que envía el mensaje espera a que un proceso lo reciba para continuar su ejecución. Por esto se suele llamar a esta técnica encuentro, o rendezvous. Dentro del paso de mensajes síncrono se engloba a la llamada a procedimiento remoto (RPC), muy popular en las arquitecturas cliente/servidor.

Veremos cómo implementar sincronización de procesos con paso de mensajes en las unidades 3, 4 y 5.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Requisitos: seguridad, vivacidad, eficiencia y reusabilidad

Como cualquier aplicación, los programas concurrentes deben cumplir una serie de requisitos de calidad. En este apartado, veremos algunos aspectos que nos permitirán desarrollar proyectos concurrentes con, casi, la completa certeza de que estamos desarrollando software de calidad.

Todo programa concurrente debe satisfacer dos tipos de propiedades:

- Propiedades de seguridad ("safety"): estas propiedades son relativas a que en cada instante de la ejecución no debe haberse producido algo que haga entrar al programa en un estado erróneo:
  - Dos procesos no deben entrar simultáneamente en una sección crítica.
  - Se respetan las condiciones de sincronismo, como: el consumidor no debe consumir el dato antes de que el productor los haya producido; y, el productor no debe producir un dato mientras que el buffer de comunicación esté lleno.
- Propiedades de vivacidad ("liveness"): cada sentencia que se ejecute conduce en algún modo a un avance constructivo para alcanzar el objetivo funcional del programa. Son, en general, muy dependientes de la política de planificación que se utilice. Ejemplos de propiedades de vivacidad son:
  - No deben producirse bloqueos activos (livelock). Conjuntos de procesos que ejecutan de forma continuada sentencias que no conducen a un progreso constructivo.
  - Aplazamiento indefinido (starvation): consiste en el estado al que puede llegar un programa que aunque potencialmente puede avanzar de forma constructiva. Esto puede suceder, como consecuencia de que no se le asigna tiempo de procesador en la política de planificación; o, porque en las condiciones de sincronización hemos establecido criterios de prioridad que perjudican siempre al mismo proceso.
  - Interbloqueo (deadlock): se produce cuando los procesos no pueden obtener, nunca, los recursos necesarios para finalizar su tarea. Vimos un ejemplo de esta situación en el apartado Condiciones de competencia.

Es evidentemente, que también nos preocuparemos por diseñar nuestras aplicaciones para que sean eficientes:

- No utilizarán más recursos de los necesarios.
- Buscaremos la rigurosidad en su implementación: toda la funcionalidad esperada de forma correcta y concreta.

Y, en cuanto a la reusabilidad, debemos tenerlo ya, muy bien aprendido:

- Implementar el código de forma modular: definiendo clases, métodos, funciones, ...
- Documentar correctamente el código y el proyecto.

Para conseguir todo lo anterior contaremos con los patrones de diseño; y pondremos especial cuidado en documentar y depurar convenientemente.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Arquitecturas y patrones de diseño

La Arquitectura del Software, también denominada arquitectura lógica, es el diseño de más alto nivel de la estructura de un sistema. Consiste en un conjunto de patrones y abstracciones coherentes con base a las cuales se pueden resolver los problemas. A semejanza de los planos de un edificio o construcción, estas indican la estructura, funcionamiento e interacción entre las partes del software.

La arquitectura de software, tiene que ver con el diseño y la implementación de estructuras de software de alto nivel. Es el resultado de ensamblar un cierto número de elementos arquitectónicos de forma adecuada para satisfacer la mayor funcionalidad y requerimientos de desempeño de un sistema, así como requerimientos no funcionales, como la confiabilidad, escalabilidad, portabilidad, y disponibilidad; mantenibilidad, auditabilidad, flexibilidad e interacción.

Generalmente, no es necesario inventar una nueva arquitectura de software para cada sistema de información. Lo habitual es adoptar una arquitectura conocida en función de sus ventajas e inconvenientes para cada caso en concreto. Así, las arquitecturas más universales son:

- Monolítica. El software se estructura en grupos funcionales muy acoplados.
- Cliente-servidor. El software reparte su carga de cómputo en dos partes independientes: consumir un servicio y proporcionar un servicio.
- Arquitectura de tres niveles. Especialización de la arquitectura cliente-servidor donde la carga se divide en capas con un reparto claro de funciones: una capa para la presentación (interfaz de usuario), otra para el cálculo (donde se encuentra modelado el negocio) y otra para el almacenamiento (persistencia). Una capa solamente tiene relación con la siguiente.

Otras arquitecturas menos conocidas son:

- En pipeline. Consiste en modelar un proceso comprendido por varias fases secuenciales, siendo la entrada de cada una la salida de la anterior.
- Entre pares. Similar a la arquitectura cliente-servidor, salvo porque podemos decir que cada elemento es igual a otro (actúa simultáneamente como cliente y como servidor).
- En pizarra. Consta de múltiples elementos funcionales (llamados agentes) y un instrumento de control o pizarra. Los agentes estarán especializados en una tarea concreta o elemental. El comportamiento básico de cualquier agente, es: examinar la pizarra, realizar su tarea y escribir sus conclusiones en la misma pizarra. De esta manera, otro agente puede trabajar sobre los resultados generados por otro.
- Orientada a servicios (Service Oriented Architecture - SOA) Se diseñan servicios de aplicación basados en una definición formal independiente de la plataforma subyacente y del lenguaje de programación, con una interfaz estándar; así, un servicio C# podrá ser usado por una aplicación Java.
- Dirigida por eventos. Centrada en la producción, detección, consumo de, y reacción a eventos.

Los patrones de diseño, se definen como soluciones de diseño que son válidas en distintos contextos y que han sido aplicadas con éxito en otras ocasiones:

- Ayudan a "arrancar" en el diseño de un programa complejo.
  - Dan una descomposición de objetos inicial "bien pensada".
  - Pensados para que el programa sea escalable y fácil de mantener.
  - Otra gente los ha usado y les ha ido bien.
- Ayudan a reutilizar técnicas.
  - Mucha gente los conoce y ya sabe como aplicarlos.
  - Están en un alto nivel de abstracción.
  - El diseño se puede aplicar a diferentes situaciones.

Existen dos modelo básicos de programas concurrentes:

- Un programa resulta de la actividad de objetos activos que interaccionan entre si directamente o a través de recursos y servicios pasivos.
- Un programa resulta de la ejecución concurrente de tareas. Cada tarea es una unidad de trabajo abstracta y discreta que idealmente puede realizarse con independencia de las otras tareas.

No es obligatorio utilizar patrones, solo es aconsejable en el caso de tener el mismo problema o similar que soluciona el patrón, siempre teniendo en cuenta que en un caso particular puede no ser aplicable.

Como podemos ver en la siguiente web, lo normal es que encontremos la definición de los patrones de diseño en UML (Unified Modeling Language – Lenguaje Unificado de Modelado).



Enlace a la web SourceMaking (en inglés), con ejemplos de patrones de diseño, implementados.

Aunque un diagrama UML es muy fácil de entender, en el siguiente enlace, encontrarás un tutorial sencillito.



Enlace a la web DocIRS, a su tutorial sobre UM

Para saber másEn 1990 se publicó un libro que recopilaba 23 patrones de diseño comunes. Este libro marcó el punto de inicio en el que los patrones de diseño tuvieron gran éxito en el mundo de la informática. En la actualidad, seguimos haciendo uso de esos 23 patrones y algunos adicionales, sobre todo relacionados con el diseño de interfaces web.

Saber más

En 1990 se publicó un libro que recopilaba 23 patrones de diseño comunes. Este libro marcó el punto de inicio en el que los patrones de diseño tuvieron gran éxito en el mundo de la informática. En la actualidad, seguimos haciendo uso de esos 23 patrones y algunos adicionales, sobre todo relacionados con el diseño de interfaces web.



Enlace a la versión on-line del libro "Design Patterns Book".

La arquitectura SOA es de gran importancia en el desarrollo de aplicaciones en el mundo empresarial, porque diseñar siguiendo esta arquitectura, permite una gran integración e interoperabilidad entre servicios y aplicaciones.

Ficheros adjuntos



- Documento pdf del modelo de referencia SOA. (Ventana nueva)

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Documentación

Para la documentación de nuestras aplicaciones tendremos en cuenta:

> **Advertencia**
>
> Hay que añadir explicaciones a todo lo que no es evidente. Pero, no hay que repetir lo que se hace, sino explicar por qué se hace.

Documentando nuestro código responderemos a estas preguntas:

- ¿De qué se encarga una clase? ¿Un paquete?
- ¿Qué hace un método? ¿Cuál es el uso esperado de un método?
- ¿Para qué se usa una variable? ¿Cuál es el uso esperado de una variable?
- ¿Qué algoritmo estamos usando? ¿De dónde lo hemos sacado?
- ¿Qué limitaciones tiene el algoritmo? ¿... la implementación?
- ¿Qué se debería mejorar ... si hubiera tiempo?

En la siguiente tabla tenemos un resumen de los distintos tipos de comentarios en Java:

Tipos de comentarios en Java

	Javadoc	Una línea	Tipo C
Sintáxis	Comienzan con `/**`, se pueden prolongar a lo largo de varias líneas (que probablemente comiencen con el carácter `*`) y terminan con los caracteres `*/`. Cuenta con etiquetas específicas tipo: `@author`, `@param`, `@return`,...	Comienzan con `//` y terminan con la línea.	Comienzan con `/*`, se pueden prolongar a lo largo de varias líneas (que probablemente comiencen con el carácter `*`) y terminan con los caracteres `*/`.
Propósito	Generar documentación externa.	Documentar código que no necesitamos que aparezca en la documentación externa.	Eliminar código.
Uso	**Obligado:** ✓ Al principio de cada clase. ✓ Al principio de cada método.	✓ Al principio de fragmento de código no evidente. ✓ A lo largo de los bucles.	Ocurre a menudo que código obsoleto no queremos que desaparezca, sino mantenerlo "por si

	✓ Antes de cada variable de clase.	✓ Siempre que hagamos algo raro o que el código no sea evidente.	acaso". Para que no se ejecute, se comenta.

Nota

Debes consultar los siguientes enlaces para completar tus conocimientos en la generación de documentación con javadoc



Documentación oficial sobre Javadoc (inglés).

Además de lo anterior, al documentar nuestras aplicaciones concurrentes destacaremos:

- Las condiciones de sincronismo que se hayan implementado en la clase o método (en la documentación javadoc).
- Destacaremos las regiones críticas que hayamos identificado y el recurso que compartido a proteger.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Dificultades en la depuración.

Cuando estamos programando aplicaciones que incluyen mecanismos de sincronización y acceden a recursos de forma concurrente junto con otras aplicaciones. A la hora de depurarlas, nos enfrentaremos a:

- Los mismos problemas de depuración de una aplicación secuencial.
- Además de nuevos errores de temporización y sincronización propios de la programación concurrente.

Los programas secuenciales presentan una línea simple de control de flujo. Las operaciones de este tipo de programas están estrictamente ordenados como una secuencia temporal lineal.

- El comportamiento del programa es solo función de la naturaleza de las operaciones individuales que constituye el programa y del orden en que se ejecutan.
- En los programas secuenciales, el tiempo que tarda cada operación en ejecutarse no tiene consecuencias sobre el resultado.
- Para validar un programa secuencial necesitaremos comprobar:
    - La correcta respuesta a cada sentencia.
    - El correcto orden de ejecución de las sentencias.

Para validar un programa concurrente se requiere comprobar los mismos aspectos que en los programas secuenciales, además de los siguientes nuevos aspectos:

- Las sentencias se pueden validar individualmente solo si no están involucradas en el uso de recursos compartidos.
- Cuando existen recursos compartidos, los efectos de interferencia entre las sentencias concurrentes pueden ser muy variados y la validación es muy difícil. Comprobaremos la corrección en la definición de las regiones críticas y que se cumple la exclusión mutua.
- Al comprobar la correcta implementación del sincronismo entre aplicaciones; que es forzar la ejecución secuencial de tareas de distintos procesos, introduciendo sentencias explícitas de sincronización. Tendremos en cuenta que el tiempo no influye sobre el resultado.

El problema es que las herramientas de depuración no nos proporcionan toda la funcionalidad que quisiéramos para poder depurar nuestros programas concurrentes.

¿Con qué herramientas contamos para depurar programas concurrentes?

- El depurador del IDE .
- Hacer volcados de actividad en un fichero de log o en pantalla de salida (nos permitirá hacernos una idea de lo que ha estado pasando durante las pruebas de depuración).
- Herramientas de depuración específicas: TotalView (SW comercial), StreamIt Debugger Tool (plugin para eclipse), ...

Una de las nuevas situaciones a las que nos enfrentamos es que a veces, los errores que parecen estar sucediendo, pueden desaparecer cuando introducimos código para tratar de identificar el

problema.

Nos damos cuenta de la complejidad que entraña depurar el comportamiento de aplicaciones concurrentes, es por ello, que al diseñarlas, tendremos en cuenta los patrones de diseño, que ya están diseñados resolviendo errores comunes de la concurrencia. Podemos verlos como 'recetas', que nos permiten resolver los problemas 'tipo' que se presentan en determinadas condiciones de sincronismo y/o en los accesos concurrentes a un recurso.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Programación paralela y distribuida

Dos procesos se ejecutan de forma paralela, si las instrucciones de ambos se están ejecutando realmente de forma simultánea. Esto sucede en la actualidad en sistemas que poseen más de un núcleo de procesamiento.

La programación paralela y distribuida consideran los aspectos conceptuales y físicos de la computación paralela; siempre con el objetivo de mejorar las prestaciones aprovechado la ejecución simultánea de tareas.

Tanto en la programación paralela como distribuida, existe ejecución simultánea de tareas que resuelven un problema común. La diferencia entre ambas es:

- La programación paralela se centra en microprocesadores multinúcleo (en nuestros PC y servidores); o ,sobre los llamados supercomputadores, fabricados con arquitecturas específicas, compuestos por gran cantidad de equipos idénticos interconectados entre sí, y que cuentan son sistemas operativos propios.
- La programación distribuida, se centra en sistemas formados por un conjunto de ordenadores heterogéneos interconectados entre sí, por redes de comunicaciones de propósito general: redes de área local, metropolitana; incluso, a través de Internet. Su gestión se realiza utilizando componentes, protocolos estándar y sistemas operativos de red.

En la computación paralela y distribuida:

- Cada procesador tiene asignada la tarea de resolver una porción del problema.
- En programación paralela, los procesos pueden intercambiar datos, a través de direcciones de memoria compartidas o mediante una red de interconexión propia.
- En programación distribuida, el intercambio de datos y la sincronización se realizará mediante intercambio de mensajes.
- El sistema se presenta como una unidad ocultando la realidad de las partes que lo forman.

Saber más

Cada 6 meses se publica un ranking con 500 supercomputadores más potentes del mundo. En noviembre de 2011 el supercomputador más ponente se encuentra en Japón y posee más de 700.000 núcleos de procesamiento. Curiosamente, en junio de 2005, el computador MareNostrum que se encuentra en Barcelona ocupó el puesto 5 del Top500, con sus nada despreciables 4800 núcleos.



Listas de rankings del Top500.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Conceptos básicos

Comencemos revisando algunas clasificaciones de sistemas distribuidos y paralelos:

- En función de los conjuntos de instrucciones y datos (conocida como la taxonomía de Flynn):
  - La arquitectura secuencial la denominaríamos SISD (single instruction single data).
  - Las diferentes arquitecturas paralelas (o distribuidas) en diferentes grupos: SIMD (single instruction multiple data), MISD (multiple instruction single data) y MIMD (multiple instruction multiple data), con algunas variaciones como la SPMD (single program multiple data).
- Por comunicación y control:
  - Sistemas de multiprocesamiento simétrico (SMP). Son MIMD con memoria compartida . Los procesadores se comunican a través de esta memoria compartida; este es el caso de los microprocesadores de múltiples núcleos de nuestros PCs.
  - Sistemas MIMD con memoria distribuida . La memoria está distribuida entre los procesadores (o nodos) del sistema, cada uno con su propia memoria local, en la que poseen su propio programa y los datos asociados. Una red de interconexión conecta los procesadores (y sus memorias locales), mediante enlaces (links) de comunicación, usados para el intercambio de mensajes entre los procesadores. Los procesadores intercambian datos entre sus memorias cuando se pide el valor de variables remotas. Tipos específicos de estos sistemas son:
    - Clusters. Consisten en una colección de ordenadores (no necesariamente homogéneos) conectados por red para trabajar concurrentemente en tareas del mismo programa. Aunque la interconexión puede no ser dedicada.
    - Grid. Es un cluster cuya interconexión se realiza a través de internet.

Veamos una introducción a algunos conceptos básicos que debemos conocer para desarrollar en sistemas distribuidos:

- Distribución: construcción de una aplicación por partes, a cada parte se le asigna un conjunto de responsabilidades dentro del sistema.
- Nudo de la red: uno o varios equipos que se comportan como una unidad de asignación integrada en el sistema distribuido.
- Un objeto distribuido es un módulo de código con plena autonomía que se puede instanciar en cualquier nudo de la red y a cuyos servicios pueden acceder clientes ubicados en cualquier otro nudo.
- Componente: Elemento de software que encapsula una serie de funcionalidades. Un componente, es una unidad independiente, que puede ser utilizado en conjunto con otros componentes para formar un sistema más complejo (concebido por ser reutilizable). Tiene especificado: los servicios que ofrece; los requerimientos que necesarios para poder ser instalado en un nudo; las posibilidades de configuración que ofrece; y, no está ligado a ninguna aplicación, que se puede instanciar en cualquier nudo y ser gestionado por herramientas automáticas. Sus características:

    - Alta cohesión: todos los elementos de un componente están estrechamente relacionados.
    - Bajo acoplamiento: nivel de independencia que un componente respecto a otros.
- Transacciones: Conjunto de actividades que se ejecutan en diferentes nudos de una plataforma distribuida para ejecutar una tarea de negocio. Una transacción finaliza cuando todas las parte implicadas clientes y múltiples servidores confirman que sus correspondientes actividades han concluido con éxito. Propiedades ACID de una transacción:
    - Atomicidad: Una transacción es una unidad indivisible de trabajo, Todas las actividades que comprende deben ser ejecutadas con éxito.
    - Congruencia: Después de que una transacción ha sido ejecutada, la transacción debe dejar el sistema en estado correcto. Si la transacción no puede realizarse con éxito, debe restaurar el sistema al estado original previo al inicio de la transacción.
    - Aislamiento: La transacciones que se ejecutan de forma concurrente no deben tener interferencias entre ellas. La transacción debe sincronizar el acceso a todos los recursos compartidos y garantizar que las actualizaciones concurrentes sean compatibles entre si.
    - Durabilidad: Los efectos de una transacción son permanentes una vez que la transacción ha finalizado con éxito.
- Gestor de transacciones. Controla y supervisa la ejecución de transacciones, asegurando sus propiedades ACID.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Tipos de paralelismo

Las mejoras arquitectónicas que han sufrido los computadores se han basado en la obtención de rendimiento explotando los diferentes niveles de paralelismo.

En un sistema podemos encontrar los siguientes niveles de paralelismo:

- A nivel de bit. Conseguido incrementando el tamaño de la palabra del microprocesador. Realizar operaciones sobre mayor número de bits. Esto es, el paso de palabras de 8 bits, a 16, a 32 y en los microprocesadores actuales de 64 bits.
- A nivel de instrucciones. Conseguida introduciendo pipeline en la ejecución de instrucciones máquina en el diseño de los microprocesadores.
  A nivel de bucle. Consiste en dividir las interaciones de un bucle en partes que se pueden realizar de manera complementaria. Por ejemplo, un bucle de 0 a 100; puede ser equivalente a dos bucles, uno de 0 a 49 y otro de 50 a 100.
- A nivel de procedimientos. Identificando qué fragmentos de código dentro de un programa pueden ejecutarse de manera simultánea sin interferir la tarea de una en la otra.
- A nivel de tareas dentro de un programa. Tareas que cooperan para la solución del programa general (utilizado en sistemas distribuidos y paralelos).
- A nivel de aplicación dentro de un ordenador. Se refiere a los conceptos que vimos al principio de la unidad, propios de la gestión de procesos por parte del sistema operativo multitarea: planificador de procesos, Round-Robin, quamtum, etc.

En sistemas distribuidos hablaremos del tamaño de grano o granularidad, que es una medida de la cantidad de computación de un proceso software. Y se considera como el segmento de código escogido para su procesamiento paralelo.

- Paralelismo de Grano Fino: No requiere tener mucho conocimiento del código, la paralelización se obtiene de forma casi automática. Permite obtener buenos resultados en eficiencia en poco tiempo. Por ejemplo: la descomposición de bucles.
- Paralelismo de Grano Grueso: Es una paralelización de alto nivel, que engloba al grano fino. Requiere mayor conocimiento del código, puesto que se paraleliza mayor cantidad de él. Consigue mejores rendimientos, que la paralelización fina, ya que intenta evitar los overhead (exceso de recursos asignados y utilizados) que se suelen producir cuando se divide el problema en secciones muy pequeñas. Un ejemplo de paralelismo grueso lo obtenemos descomponiendo el problema en dominios (dividiendo conjunto de datos a procesar, acompañando a estos, las acciones que deban realizarse sobre ellos).

> ### Saber más
>
> En programación paralela, además de utilizar CPUs, se utilizan GPU (las unidades de procesamiento de las tarjetas gráficas). Las GPU poseen una mayor potencia de

cómputo al ser procesadores específicamente diseñados para resolver tareas intensivas de cómputo en tiempo real de direccionamiento de gráficos en 3D de alta resolución. En general, tienen muy buen comportamiento en algoritmos de ordenación rápida de grandes listas de datos, y transformaciones bidimensionales.

El proyecto de arquitectura de computación paralela con GPUs CUDA, de NVidia, puede programarse en multitud de lenguajes de programación.

Por ejemplo, las simulaciones de dinámica molecular obtienen mejores resultados en sistemas CUDA.



Proyecto CUDA de NVidia.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Modelos de infraestructura para programación distribuida

Las aplicaciones distribuidas requieren que componentes que se ejecutan en diferentes procesadores se comuniquen entre sí. Los modelos de infraestructura que permiten la implementación de esos componentes, son:

- Uso de Sockets: Facilitan la generación dinámica de canales de comunicación. Es actualmente la base de la comunicación. Pero al ser de muy bajo nivel de abstracción, no son adecuados a nivel de aplicación.
- Remote Procedure Call (RPC): Abstrae la comunicación a nivel de invocación de procedimientos. Es adecuada para programación estructurada basada en librerías.
- Invocación remota de objetos: Abstrae la comunicación a la invocación de métodos de objetos que se encuentran distribuidos por el sistema distribuido. Los objetos se localizan por su identidad. Es adecuada para aplicaciones basadas en el paradigma OO.
- RMI (Remote Method Invocation) es la solución Java para la comunicación de objetos Java distribuidos. Presenta un inconveniente, y es el paso de parámetros por valor implica tiempo para hacer la serialización, enviar los objetos serializados a través de la red y luego volver a recomponer los objetos en el destino.
- CORBA (Common Object Request Brocker Architecture) . Para facilitar el diseño de aplicaciones basadas en el paradigma Cliente/Servidor. Define servidores estandarizados a través de un modelo de referencia, los patrones de interacción entre clientes y servidores y las especificaciones de las APIs.
- MPI ("Message Passing Interface", Interfaz de Paso de Mensajes) es un estándar que define la sintaxis y la semántica de las funciones contenidas en una biblioteca de paso de mensajes diseñada para ser usada en programas que exploten la existencia de múltiples procesadores.
- La Interfaz de Paso de Mensajes es un protocolo de comunicación entre computadoras. Es el estándar para la comunicación entre los nodos que ejecutan un programa en un sistema de memoria distribuida.
    - Las llamadas de MPI se dividen en cuatro clases:Llamadas utilizadas para inicializar, administrar y finalizar comunicaciones.
    - Llamadas utilizadas para transferir datos entre un par de procesos.
    - Llamadas para transferir datos entre varios procesos.
    - Llamadas utilizadas para crear tipos de datos definidos por el usuario.
- Máquina paralela virtual. Paquetes software que permite ver el conjunto de nodos disponibles como una máquina virtual paralela ; ofreciendo una opción práctica, económica y popular hoy en día para aproximarse al cómputo paralelo.

Saber más

Existen gran cantidad de proyectos computación distribuida. En el siguiente enlace, puedes ver un listado de ellos. Por curiosidad, dos se están desarrollando en la

Univerdidad Complutense de Madrid. Uno de ellos es una red neuronal que simula el comportamiento de la gran y compleja red de células neuronales autómatas; otro proyecto, utiliza modelos matemáticos de una red social, para estudiar la evolución ideológica de un grupo de personas a través del tiempo.



Lista de proyectos de computación distribuida.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0